

RUMELİ ÜNİVERSİTESİ

Mühendislik ve Doğa Bilimleri Fakültesi - Bilgisayar Mühendisliği

BLG462 - BÜYÜK VERİ

DÖNEM PROJESİ RAPORU

TWİTTER VERİSİ İLE UÇTAN UCA BÜYÜK VERİ PIPELİNE'İ

Ad Soyad:	Onur Aydı
Öğrenci No:	221201011
Ders:	BLG462- Büyük Veri
Akademik Yıl:	2025-2026
Dönem:	Bahar Dönemi
Repo:	https://github.com/onuraydi/BigData-twitter-pipeline



Teslim Tarihi: 10 Haziran 2026

İÇİNDEKİLER

TWİTTER VERİSİ İLE UÇTAN UCA BÜYÜK VERİ PİPELİNE'İ	1
1. GİRİŞ:.....	2
2. Projenin Kapsamı ve Amacı:	3
3. Veri Temini ve Keşif:	3
3.1 Veri Temini:	3
3.2 Veri Seti Keşfi:.....	4
3.3 DIKW Piramidi Çerçevesinde Veri Değerlendirmesi:	5
3.4 Veri Seti Yapısal Analizi ve İstatistiksel Çıkarımlar:	5
4. Veri Temizleme ve Hazırlık:	7
4.1 Gürültü Arındırma ve Metin Standardizasyonu:	7
4.2 Eksik Veri (NaN/Null) Yönetimi:	8
4.3 Duplicate Kayıtların Yönetilmesi:.....	9
4.4 Tip Dönüşümü:	9
4.5 Veri Kalitesi Boyutları ve Karşılaştırmalı Raporlama:.....	9
5. MongoDB Entegrasyonu ve NoSQL Sorguları:	11
5.1 Veri Tabanı Bağlantısı ve Sistem Konfigürasyonu:	11
5.2 NoSQL Şema Tasarımı:	12
5.3 Verilerin MongoDB'ye Aktarılması:.....	12
5.4 Sorgular ve Veri Tabanı İşlemleri:	13
5.4.1 Find() Metodu Sorguları:.....	13
5.4.2 Aggregation Pipeline İşlemleri:	19
5.4.3 Index İşlemleri:	24
5.4.4 Update İşlemleri:.....	27
5.4.5 Delete İşlemleri:.....	29
5.4.6 İleri Düzey Sorgular:.....	30
6. PySpark İle Dağıtık Veri İşleme:	34
6.1. Verinin PySpark'a Yüklenmesi:	34
6.2. Dağıtık Veri Dönüşüm Operasyonları (Transformations):	36
6.3 Dağıtık Veri Eylem (Action) Operasyonları:.....	39
6.4 Dağıtık Spark SQL Kullanımı:	41

6.5 Pandas ve PySpark Karşılaştırması:	44
7. Veri Görselleştirme:	45
8. ETL Diyagramı ve Dokümantasyon:	51
8.1 Uçtan Uca ETL Mimari Akış Diyagramı:	51
8.2 ETL ve ELT Yaklaşımlarının Karşılaştırması:	52
8.3 Proje Mimarisinin Gerekçelendirilmesi (Neden ETL?)	52

1. GİRİŞ:

Günümüzde sosyal medya platformları, yapılandırılmamış (unstructured) verinin en büyük kaynaklarından birini oluşturmaktadır. Özellikle Twitter gibi mikroblog platformları üzerinden akan veriler; hacim (volume), hız (velocity) ve çeşitlilik (variety) bakımından Büyük Veri'nin karakteristik özelliklerini eksiksiz şekilde taşımaktadır. Bu verilerin işlenmesi, anlamlandırılması ve stratejik karar destek mekanizmalarına dönüştürülmesi, geleneksel veri tabanı sistemlerinin ötesinde dağıtık mimariler ve NoSQL çözümleri gerektirmektedir.

Bu proje kapsamında, Amerika havayolu şirketlerine yönelik müşteri geri bildirimlerini içeren bir veri seti kullanılarak, uçtan uca bir büyük veri pipeline'ı tasarlanmıştır. Proje; verinin ham haliyle temininden başlayarak, temizlenmesi, NoSQL tabanlı modellenmesi, dağıtık sistemlerde analiz edilmesi ve görselleştirilmesine kadar olan tüm ETL (Extract, Transform, Load) süreçlerini kapsayan modüler bir mimari sunmaktadır.

2. Projenin Kapsamı ve Amacı:

Projenin temel amacı, büyük veri yaşam döngüsünün her aşamasını operasyonel bir düzlemde uygulamaktır. Bu doğrultuda aşağıdaki alt amaçlar hedeflenmiştir:

- **Veri Kalitesinin Sağlanması:** Ham metin verilerindeki gürültülerin (URL, hashtag, özel karakterler) temizlenerek verinin analiz edilebilir hale getirilmesi.
- **NoSQL Veri Modelleme:** ilişkisel olmayan veri tabanlarında (MongoDB) esnek şema tasarımı yapılarak yüksek performanslı sorgulama yeteneklerinin kazanılması.
- **Dağıtık Veri İşleme:** PySpark mimarisi kullanılarak, verinin paralel işleme teknikleriyle nasıl ölçeklendirilebileceğinin simüle edilmesi.
- **Veri Görselleştirme ve Dokümantasyon:** Temizlenmiş veriler üzerinden duygu analizi (sentiment analysis) ve istatistiksel çıkarımlar yaparak veriyi görselleştirmek ve dokümante etmek.

3. Veri Temini ve Keşif:

Bu aşamada projenin ilk aşaması olan veriyi temin etme ve keşfetme aşamasındaki çıktıları içermektedir.

3.1 Veri Temini:

İlgili link üzerinden veri setine ulaşılmış olup mevcutta kaggle üzerinde bir hesaba sahip olduğum için direkt olarak veri setini bilgisayarıma indirdim. İçerisindeki 'Tweets.csv' dosyasını

projemdeki 'data/raw' klasörü içerisine taşıdım. Ardından ilgili değişikliği GitHub repoma pushladım. Sonuç olarak veri proje içinde kullanıma hazır hale gelmiş oldu.

3.2 Veri Seti Keşfi:

Bir önceki adımda temin edilen veri seti üzerinde, veri setinin çeşitli özelliklerinin öğrenilmesi için çeşitli işlemler yapıldı. Bu işlemler sonucu şu bilgiler elde edildi:

- **Veri Setinin Boyutu:** Yapılan işlem sonucu veri setinin boyutunun **(14640, 15)** olduğu öğrenildi. Burada 15 sütun, 14640 satır sayısıdır.
- **Sütun İsimleri ve Veri Tipi:** Veri setindeki her sütunun ismi ve bu sütunların veri tipi öğrenilmiştir. Bu isimler ve veri tipleri şöyledir:

```
--- Veri Tipleri ---
tweet_id                int64
airline_sentiment        object
airline_sentiment_confidence float64
negativereason           object
negativereason_confidence float64
airline                  object
airline_sentiment_gold    object
name                     object
negativereason_gold       object
retweet_count            int64
text                     object
tweet_coord              object
tweet_created             object
tweet_location            object
user_timezone             object
dtype: object
```

- **Örnek Kayıtlar:** Veri seti içerisindeki kayıtlardan 10 tanesi rastgele bir şekilde çekilerek örnek veri incelemesi yapılmıştır. Rastgele çekilen örnek verilerden birkaç tanesi şu şekildedir:

```
--- Örnek Kayıtlar ---
tweet_id  airline_sentiment  airline_sentiment_confidence  negativereason  ...  tweet_coord  tweet_created  tweet_location  user_timezone
7079  56993949465462272      negative                      1.0000          Late Flight  ...  NaN  2015-02-23 11:18:59 -0800  NaN  Eastern Time (US & Canada)
2187  569323727818657792      negative                      0.6737          Bad Flight  ...  NaN  2015-02-21 18:32:08 -0800  NaN  NaN
7304  569676138253049859      positive                     0.6838          NaN  ...  NaN  2015-02-22 17:52:30 -0800  New York  Eastern Time (US & Canada)
5808  568582374549807104      negative                     1.0000  Customer Service Issue  ...  NaN  2015-02-19 17:26:16 -0800  NaN  Quito
1491  569699471954587648      negative                     1.0000          Late Flight  ...  NaN  2015-02-22 19:25:13 -0800  ÜT: 33.987103,-118.399024  NaN
1865  569544873809989632      positive                     1.0000          NaN  ...  NaN  2015-02-22 09:10:54 -0800  NaN  Central Time (US & Canada)
8834  567808748711051264      neutral                     1.0000          NaN  ...  [0.0, 0.0]  2015-02-17 14:12:09 -0800  Wayne, NJ  Eastern Time (US & Canada)
222  569222494952886272      positive                     0.6522          NaN  ...  NaN  2015-02-21 11:49:53 -0800  NaN  NaN
11446  568111010856480768      negative                     1.0000          Can't Tell  ...  NaN  2015-02-18 10:13:14 -0800  NY  NaN
7958  568929608566837249      positive                     1.0000          NaN  ...  NaN  2015-02-20 16:26:03 -0800  Hastings on Hudson, NY  Eastern Time (US & Canada)
```

- **Eksik ve Tekrarlayan Veri Analizi:** Veri seti kapsamındaki tüm değişkenler (sütunlar) taranarak eksik değer barındıranlar tespit edilmiştir. Bu doğrultuda, her bir değişkene ait eksik veri adetleri ve bunların toplam veri içindeki yüzdesel oranları hesaplanmıştır. Ek olarak, veri setindeki mükerrer (tekrar eden) kayıtların sayısı belirlenmiştir. Gerçekleştirilen bu analizlerin sonuçları aşağıda sunulmuştur:

```
--- Eksik Bilgiler ---
Missing_Count Missing_Percentage
negativereason_gold 14608 99.781421
airline_sentiment_gold 14600 99.726776
tweet_coord 13621 93.039617
negativereason 5462 37.308743
user_timezone 4820 32.923497
tweet_location 4733 32.329235
negativereason_confidence 4118 28.128415

--- Tekrarlayan kayıtlar ---
36
```

3.3 DIKW Piramidi Çerçevesinde Veri Değerlendirmesi:

Büyük Veri temel süreçlerinden biri de karmaşık veri yığınlarını karar destek mekanizmaları için uygun hale dönüştürmektir. Bu kapsamda “Bu ham veriden hangi bilgiye ulaşılabilir?” sorusu DIKW (Data, Information, Knowledge, Wisdom) hiyerarşisi üzerinden şu şekilde yanıtlanmaktadır:

- **Veri (Data):** Kaggle platformundan temin edilen ve 14.640 kayıttan oluşan bu katman; ham tweet metinleri, zaman damgaları ve kullanıcı kimlik (ID) değerlerini içermektedir. Veri seti; URL'ler, noktalama işaretleri ve özel karakterler barındırması sebebiyle "yapılandırılmamış veri" (unstructured data) kategorisinde sınıflandırılmaktadır. Bu nedenle, mevcut ham formuyla doğrudan analize dahil edilmesi veya anlamlı bir eylem planı sunması mümkün değildir.
- **Information (Bilgi):** Ham verinin, uygulanan veri temizleme (preprocessing) algoritmalarıyla yapılandırılmış (structured) bir formata dönüştüğü aşamadır. Gürültüden arındırılmış bir tweet'in 'positive', 'negative' veya 'neutral' etiketleriyle (sentiment) sınıflandırılıp spesifik bir havayolu şirketiyle eşleştirilmesi veriye bağlam kazandırır.
- **Bilgi (Knowledge):** Yapılandırılmış veriler üzerinde gerçekleştirilen kümeleme (aggregation) işlemleri ve NoSQL sorguları aracılığıyla gizli istatistiksel örüntülerin (patterns) ortaya çıkarıldığı katmandır. Proje kapsamında, negativereason (negatif geri bildirim sebebi) sütunu ve duygu dağılımları analiz edilerek anlamlı bağlantılar keşfedilir. Örneğin; "A havayolu şirketine yönelik negatif bildirimlerin %65'inin rötarlı uçuşlardan kaynaklandığı" veya "B havayolu şirketinin müşteri hizmetleri süreçlerinde kronik sorunlar yaşadığı" gibi somut analitik çıkarımlar bu katmanda elde edilir.
- **Wisdom (Bilgelik):** Analiz edilen bilgi birikiminin eyleme dönüştürülebilir stratejik kararlara zemin oluşturduğu nihai aşamadır. Elde edilen istatistiksel kanıtlara dayanarak havayolu yönetiminin; uçuş operasyonlarını optimize etmesi, şikayetlerin yoğun olduğu saat dilimlerinde müşteri hizmetleri kapasitesini artırması veya bagaj yönetim süreçlerini yeniden tasarlaması yönünde alacağı proaktif kararlardır.

3.4 Veri Seti Yapısal Analizi ve İstatistiksel Çıkarımlar:

İlk aşamanın son adımı olarak; veri setinin yapısal ve istatistiksel özelliklerini incelemek amacıyla Pandas kütüphanesine ait 'info()' ve 'describe()' metotları kullanılmıştır. Bu analiz sonucunda elde edilen istatistiksel özetler ve teknik bulgular, veri temizleme (data cleaning) operasyonları ile NoSQL veri modelleme mimarisinde izlenecek stratejilerin belirlenmesinde kritik bir temel oluşturmuştur.

3.4.1 info() metodu:

İlgili metot (info()) aracılığıyla veri setinin yapısal özellikleri kapsamlı bir şekilde incelenmiştir. Bu analiz sonucunda aşağıdaki temel bilgiler doğrulanmıştır:

- **Toplam Kayıt Sayısı:** Veri setinin 14.640 kayıttan (satırdan) oluştuğu teyit edilmiştir.
- **Değişken Yapısı ve Veri Tipleri:** Veri setinde yer alan tüm sütunlar ve bu sütunların barındırdığı veri tipleri (data types) kontrol edilmiştir.
- **Bellek Tüketimi:** Veri setinin sistem belleğinde (RAM) kapladığı alan tespit edilmiştir.

İlgili metoda ait konsol çıktısı aşağıda sunulmuştur:

```
--- Veri Seti Yapısal Bilgisi (info) ---
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 14640 entries, 0 to 14639
Data columns (total 15 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   tweet_id                             14640 non-null  int64
1   airline_sentiment                    14640 non-null  object
2   airline_sentiment_confidence         14640 non-null  float64
3   negativereason                       9178 non-null   object
4   negativereason_confidence            10522 non-null  float64
5   airline                              14640 non-null  object
6   airline_sentiment_gold                40 non-null     object
7   name                                 14640 non-null  object
8   negativereason_gold                   32 non-null     object
9   retweet_count                        14640 non-null  int64
10  text                                 14640 non-null  object
11  tweet_coord                           1019 non-null   object
12  tweet_created                         14640 non-null  object
13  tweet_location                        9907 non-null   object
14  user_timezone                         9820 non-null   object
dtypes: float64(2), int64(2), object(11)
memory usage: 10.0 MB
```

3.4.2 describe() metodu:

Nicel ve nitel verilerin istatistiksel özetini sunan describe() metodu sonucu, şu çıktılar elde edilmiştir:

- **Sınıf Dengesi (Sentiment Imbalance):** Veri setindeki en sık tekrar eden duygu durumunun 'Negative' (9178 adet) olduğu görülmüştür. Bu, müşterilerin havayolu şirketlerine genellikle şikâyet odaklı yaklaştığını ve duygu analizi modelimizin negatif sınıfa daha duyarlı olacağını göstermektedir.
- **Operasyonel Kök Nedenler:** Negatif yorumlar arasında en belirgin kategorinin "Customer Service Issue" (2.910 adet) olduğu tespit edilmiştir. Bu veri, projenin "Bilgi Birikimi" (Knowledge) katmanında havayolu şirketlerine verilecek stratejik tavsiyelerin temelini oluşturur.

- **Pazar Payı ve Temsiliyet:** Veri setinde en çok etkileşim alan şirketin United (3.822 tweet) olduğu saptanmıştır.
- **Güven Aralığı:** 'airline_sentiment_confidence' değerinin ortalamasının %90 (0.90) olması, verideki etiketlerin yüksek doğruluk oranına sahip olduğunu ve temizleme sonrası elde edilecek sonuçların güvenilir olacağını kanıtlamaktadır.

İlgili metoda ait konsol çıktısı aşağıda sunulmuştur:

```

--- Temel İstatistiksel Dağılımlar (describe) ---

```

	count	unique	top	freq	mean	std	min	25%	50%	75%	max
tweet_id	14640.0	NaN	NaN	NaN	569218351767499200.0	779111158481835.875	567588278875213824.0	568559178101439488.0	569477857923110912.0	569890473289000960.0	570310600460525568.0
airline_sentiment	14640	3	negative	9178	NaN	NaN	NaN	NaN	NaN	NaN	NaN
airline_sentiment_confidence	14640.0	NaN	NaN	NaN	0.908169	0.16283	0.335	0.6923	1.0	1.0	1.0
negative_reason	9178	10	Customer Service Issue	2910	NaN	NaN	NaN	NaN	NaN	NaN	NaN
negative_reason_confidence	10522.0	NaN	NaN	NaN	0.638298	0.33844	0.0	0.3686	0.6786	1.0	1.0
airline	14640	6	United	3822	NaN	NaN	NaN	NaN	NaN	NaN	NaN
airline_sentiment_gold	40	3	negative	32	NaN	NaN	NaN	NaN	NaN	NaN	NaN
name	14640	7701	JetBlueNews	63	NaN	NaN	NaN	NaN	NaN	NaN	NaN
negative_reason_gold	32	13	Customer Service Issue	12	NaN	NaN	NaN	NaN	NaN	NaN	NaN
retweet_count	14640.0	NaN	NaN	NaN	0.08265	0.745778	0.0	0.0	0.0	0.0	44.0
text	14640	14427	@united thanks	6	NaN	NaN	NaN	NaN	NaN	NaN	NaN
tweet_coord	1019	832	[0.0, 0.0]	164	NaN	NaN	NaN	NaN	NaN	NaN	NaN
tweet_created	14640	14247	2015-02-24 09:54:34 -0800	5	NaN	NaN	NaN	NaN	NaN	NaN	NaN
tweet_location	9907	3081	Boston, MA	157	NaN	NaN	NaN	NaN	NaN	NaN	NaN
user_timezone	9820	85	Eastern Time (US & Canada)	3744	NaN	NaN	NaN	NaN	NaN	NaN	NaN

PS C:\Users\OMUR\Desktop\big-data-project>

4. Veri Temizleme ve Hazırlık:

Veri setinin Kaggle üzerinden temin edilip profilinin çıkarılmasının ardından, ham veriyi büyük veri analiz süreçlerine (NoSQL modelleme ve dağıtık işleme) uygun hale getirmek amacıyla veri ön işleme adımları uygulanmıştır. Bu aşamadaki tüm operasyonlar, performans kaybı yaşamamak adına Python'ın Pandas kütüphanesi kullanılarak gerçekleştirilmiştir.

4.1 Gürültü Arındırma ve Metin Standardizasyonu:

Veri seti üzerinde daha verimli işlem yapabilmek ve modelleme aşamasına temiz bir veri sunmak için yapısal gürültüler giderilmiştir. Pandas kütüphanesindeki 'str.replace()' fonksiyonu ve Regex (Düzenli İfadeler) kuralları kullanılarak metinlerdeki URL'ler, mention'lar, hashtag'ler ve özel karakterler temizlenmiştir. Son adım olarak, veri setinde standardizasyon sağlamak amacıyla tüm metinler küçük harfe çevrilmiştir.

Bu işlem için kullanılan python kodu ve kod sonucu oluşan çıktı aşağıdaki gibidir: Kod:

```

1. # regex ile veri içerisindeki url, mention, hashtag, özel karakter ve boşlukları temizler.
2. def clean_text_data(self) -> None:
3.     regex_url = r'http\S+|www\S+|https\S+'
4.     regex_mention = r'@\w+'
5.     regex_hashtag = r'#\w+'
6.     regex_special = r'^a-zA-Z0-9\s'
7.
8.     df = self.cleaned_df['text']
9.
10.    df = df.str.replace(regex_url, '', regex=True)

```



```

11. df = df.str.replace(regex_mention, '', regex=True)
12. df = df.str.replace(regex_hashtag, '', regex=True)
13. df = df.str.replace(regex_special, '', regex=True)
14. df = df.str.replace(r'\s+', ' ', regex=True).str.strip()
15. df = df.str.lower()
16.
17. self.cleaned_df['text'] = df

```

Çıktı:

```

PS C:\Users\ONUR\Desktop\big-data-project> py main.py

--- Temizleme ÖNCESİ Örnek Metinler ---
tweet_id      text
0  570306133677760513  @VirginAmerica What @dhepburn said.
1  570301130888122368  @VirginAmerica plus you've added commercials t...
2  570301083672813571  @VirginAmerica I didn't today... Must mean I n...
3  570301031407624196  @VirginAmerica it's really aggressive to blast...
4  570300817074462722  @VirginAmerica and it's a really big bad thing...

--- Temizleme SONRASI Örnek Metinler ---
tweet_id      text
0  570306133677760513  what said
1  570301130888122368  plus youve added commercials to the experience...
2  570301083672813571  i didnt today must mean i need to take another...
3  570301031407624196  its really aggressive to blast obnoxious enter...
4  570300817074462722  and its a really big bad thing about it
PS C:\Users\ONUR\Desktop\big-data-project>

```

4.2 Eksik Veri (NaN/Null) Yönetimi:

Sistemdeki bütünlüğü sağlamak adına eksik veriler hibrit bir strateji ile yönetilmiştir. Yapılan analizde, eksiklik oranı veri setinin %50'sini aşan 'airline_sentiment_gold', 'negativereason_gold', 'tweet_coord' sütunları analitik değerler taşımadıkları için veri setinden tamamen silinmiştir. Kalan Sütunlardaki yapısal eksiklikler ise kategorik veriler için 'Unknown', sayısal veriler için ise 'medyan' değerleri atanarak doldurulmuştur.

Bu işlem için kullanılan python kodu ve konsol çıktısı aşağıdaki gibidir:
Kod:

```

1. def handle_missing_value(self) -> None:
2. threshold = len(self.cleaned_df) * 0.5
3. cols_to_drop = self.cleaned_df.columns[self.cleaned_df.isnull().sum() >
threshold].tolist()
4. self.cleaned_df.drop(columns=cols_to_drop, inplace=True)
5. self.metrics["dropped_columns"] = cols_to_drop
6.
7. categorical_cols =
self.cleaned_df.select_dtypes(include=['object']).columns
8. self.cleaned_df[categorical_cols] =
self.cleaned_df[categorical_cols].fillna('Unknown')
9.
10. numeric_cols = self.cleaned_df.select_dtypes(include=['float64',
'int64']).columns
11. for col in numeric_cols:
12. self.cleaned_df[col] =
self.cleaned_df[col].fillna(self.cleaned_df[col].median())

```

Çıktı:

```
--- Eksik Veri Yönetimi ---  
Kaldırılan Sütunlar (Düşük Tamlık): ['airline_sentiment_gold', 'negativereason_gold', 'tweet_coord']
```

4.3 Duplicate Kayıtların Yönetilmesi:

Veri setinde yer alan ve istatistiksel sapmalara (bias) yol açabilecek tekrarlı kayıtlar tespit edilmiştir. Pandas'ın `drop_duplicates()` metodu uygulanarak bu veriler sistemden uzaklaştırılmıştır.

Bu işlem için kullanılan python kodu ve çıktısı aşağıdaki gibidir:

Kod:

```
1. # duplicate dataları siler.  
2. def remove_duplicates(self) -> None:  
3. self.cleaned_df.drop_duplicates(inplace=True)
```

Çıktı:

```
PS C:\Users\ONUR\Desktop\big-data-project> py main.py  
--- Temizleme Öncesi ve Sonrası Kayıt Sayısı ---  
Başlangıç: 14640  
Bitiş: 14604  
Silinen Mükerrer Satır: 36
```

4.4 Tip Dönüşümü:

Veri üzerinde işlem yapabilmek için başlangıçta 'object' türündeki 'tweet_created' sütunu 'pd.to_datetime' metodu kullanarak 'datetime' türüne dönüştürülmüştür.

Bu işlem için kullanılan python kodu ve çıktısı aşağıdaki gibidir:

Kod:

```
1. # datadaki tarih alanını datetime formatına çeviriyor.  
2. def convert_data_types(self) -> None:  
3. if 'tweet_created' in self.cleaned_df.columns:  
4. self.cleaned_df['tweet_created'] =  
    pd.to_datetime(self.cleaned_df['tweet_created'], errors='coerce')
```

Çıktı:

```
PS C:\Users\ONUR\Desktop\big-data-project> py main.py  
--- Dönüşüm ÖNCESİ Veri Tipi ---  
tweet_created sütun tipi: object  
   tweet_id      tweet_created  
0  57030613367760513  2015-02-24 11:35:52 -0800  
1  570301130888122368  2015-02-24 11:15:59 -0800  
2  570301083672813571  2015-02-24 11:15:48 -0800  
  
--- Dönüşüm SONRASI Veri Tipi ---  
tweet_created sütun tipi: datetime64[ns, UTC-08:00]  
   tweet_id      tweet_created  
0  57030613367760513  2015-02-24 11:35:52-08:00  
1  570301130888122368  2015-02-24 11:15:59-08:00  
2  570301083672813571  2015-02-24 11:15:48-08:00  
PS C:\Users\ONUR\Desktop\big-data-project>
```

4.5 Veri Kalitesi Boyutları ve Karşılaştırmalı Raporlama:

Uygulanan veri temizleme ardışık düzeni (pipeline) sonucunda, kayıt sayılarındaki değişim ve sistemin ulaştığı veri kalitesi boyutları aşağıdaki metriklerle belgelenmiştir:

- **Kayıt Sayısı Karşılaştırması:** Temizleme işlemi öncesi 14.640 olan toplam satır sayısı, 36 adet mükerrer kaydın tespit edilip silinmesiyle 14.604 tekil kayda inmiştir
- **Tamlık (Completeness):** İşlem öncesi %71,78 olan yapısal doluluk oranı, kullanılamaz sütunların silinmesi ve eksik verilerin doldurulması ile %100.00 seviyesine ulaşmıştır.
- **Tutarlılık (Consistency):** Zaman formatının metinden tarih nesnesine (datetime) geçirilmesiyle tarihsel bütünlük ve tutarlılık %100.00 olarak güvence altına alınmıştır.
- **Doğruluk (Accuracy):** Metinlerin URL, HTML tagleri ve anlamsız özel karakterlerden tamamen (%100) arındırıldığı kanıtlanmıştır.

Bu işlem için kullanılan python kodu ve çıktıları aşağıdaki gibidir:

Kod:

```

1. # data kalitesini ölçme
2. def evaluate_data_quality(self) -> None:
3.     initial_nulls = self.raw_df.isnull().sum().sum()
4.     initial_total = self.raw_df.size
5.     final_nulls = self.cleaned_df.isnull().sum().sum()
6.     final_total = self.cleaned_df.size
7.
8.     completeness_before = ((initial_total - initial_nulls) / initial_total) *
100
9.     completeness_after = ((final_total - final_nulls) / final_total) * 100
10.
11.     valid_dates = self.cleaned_df['tweet_created'].notna().sum() if
'tweet_created' in self.cleaned_df.columns else 0
12.     consistency = (valid_dates / len(self.cleaned_df)) * 100 if
len(self.cleaned_df) > 0 else 0
13.
14.     url_count =
self.cleaned_df['text'].str.contains(r'http\S+|www\S+|https\S+',
regex=True).sum()
15.     accuracy = ((len(self.cleaned_df) - url_count) / len(self.cleaned_df))
* 100
16.
17.
18.     self.metrics["quality_dimensions"] = {
19.         "Tamlık": f"%{completeness_after:.2f} (Öncesi:
%{completeness_before:.2f})",
20.         "Tutarlılık": f"%{consistency:.2f} (Tarih formatı yapısal bütünlüğü)",
21.         "Doğruluk": f"%{accuracy:.2f} (URL ve gürültüden arındırılmış metin
oran)"
22.     }
23.
24.     def execute_pipeline(self) -> pd.DataFrame:
25.         self.remove_duplicates()
26.         self.handle_missing_value()
27.         self.clean_text_data()
28.         self.convert_data_types()
29.
30.         self.metrics["after_count"] = len(self.cleaned_df)
31.         self.evaluate_data_quality()

```

```
32.  
33. return self.cleaned_df
```

Çıktılar:

```
PS C:\Users\ONUR\Desktop\big-data-project> py main.py  
--- Temizleme Öncesi ve Sonrası Kayıt Sayısı ---  
Başlangıç: 14640  
Bitiş: 14604  
Silinen Mükerrer Satır: 36  
  
--- Eksik Veri Yönetimi ---  
Kaldırılan Sütunlar (Düşük Tamlık): ['airline_sentiment_gold', 'negativereason_gold', 'tweet_coord']  
  
--- Veri Kalitesi Boyutları ---  
{  
  "Tamlık": "%100.00 (Öncesi: %71.78)",  
  "Tutarlılık": "%100.00 (Tarih formatı yapısal bütünlüğü)",  
  "Doğruluk": "%100.00 (URL ve gürültüden arındırılmış metin oran)"  
}  
  
--- Temizlenmiş Veri Örneği ---  
      tweet_id      text      tweet_created  
14510  569605359406485504  been waiting in line for over an hour in san a... 2015-02-22 13:11:15-08:00  
8518  568196466503770112  the big board says we arent leaving until afte... 2015-02-18 15:52:48-08:00  
2414  569219701001179138  but its doubtful hell fly united again and he ... 2015-02-21 11:38:46-08:00  
13098  569940740454936576  i love very much your planes can you please fo... 2015-02-23 11:23:56-08:00  
533  570291813912858625  thats weak see ya hey 2015-02-24 10:38:58-08:00  
11296  568230023251042304  delays to the max 2015-02-18 18:06:09-08:00  
7100  569937408013176832  our fleets on fleek this is what i hate 2015-02-23 11:10:41-08:00  
5173  569282355904548864  thank you ah still wish someone picked up the ... 2015-02-21 15:47:45-08:00  
8969  570309000279023616  fortunately you have staff like lynn s and dee... 2015-02-24 11:47:16-08:00  
8003  568888167681695744  thanks for your reply it was flight 1572 2015-02-20 13:41:23-08:00  
PS C:\Users\ONUR\Desktop\big-data-project>
```

5. MongoDB Entegrasyonu ve NoSQL Sorguları:

Veri setinin temizlenme ve ön işleme adımlarının tamamlanmasının ardından, projenin kalıcılık ve analitik sorgulama katmanı olan NoSQL veri tabanı aşamasına geçilmiştir. Bu bağlamda, dağıtık sistem mimarilerine ve esnek şema yapılarına uygunluğu sebebiyle belge yönelimli bir veri tabanı olan MongoDB kullanılmıştır.

5.1 Veri Tabanı Bağlantısı ve Sistem Konfigürasyonu:

Veri tabanı operasyonları için python'ın resmi MongoDB kütüphanesi olan 'pymongo' kütüphanesi kullanılmıştır. Geliştirme local mongoDB sunucusu üzerinden yürütülmüş olup, bağlantı yapılandırması 'MongoDbManager' isimli sınıf üzerinden encapsulation tekniği ile yönetilmiştir. Bu sayede veri tabanı bağlantısının sürdürülebilirliği güvence altına alınmıştır.

Kod:

```
class MongoDBManager:  
    def __init__(self, uri: str = 'mongodb://localhost:27017', db_name:  
str = 'airline_twitter_db', col_name: str = 'tweets'):  
        self.client = MongoClient(uri)  
        self.db = self.client[db_name]  
        self.collection = self.db[col_name]
```

5.2 NoSQL Şema Tasarımı:

İlişkisel veri tabanlarının aksine MongoDB, şemasız bir yapı sunar. Ancak büyük veri analitiğinde performans kayıplarını önlemek adına, temizlenen veri seti doğrudan düz bir collection olarak aktarılmamış; bunun yerine iç içe doküman şeması tasarlanmıştır.

Bu tasarımda tweet dokümanı; 'airline_info', 'feedback' ve 'user' gibi mantıksal alt objelere bölünmüştür. Bu yapı sistemi ilerleyen aşamalarında karmaşık aggregate sorgularının çok daha performanslı çalışmasını sağlayacaktır.

5.3 Verilerin MongoDB'ye Aktarılması:

Pandas DataFrame formatındaki temizlenmiş veri, oluşturulan iç içe JSON formatına uygun dictionary yapılarına dönüştürülmüştür. Veri yazma hızını maksimize etmek ve kolay yönetmek adına kayıtlar tek tek değil 'insert_many()' metodu ile toplu şekilde veri tabanına aktarılmıştır. İşlem sonucunda kayıtların başarılı şekilde mongoDB'ye aktarıldığı da teyit edilmiştir.

Kod:

```
def prep_and_bulk_insert(self, df: pd.DataFrame) -> int:
    self.collection.drop()

    df_cleaned = df.where(pd.notnull(df), None)

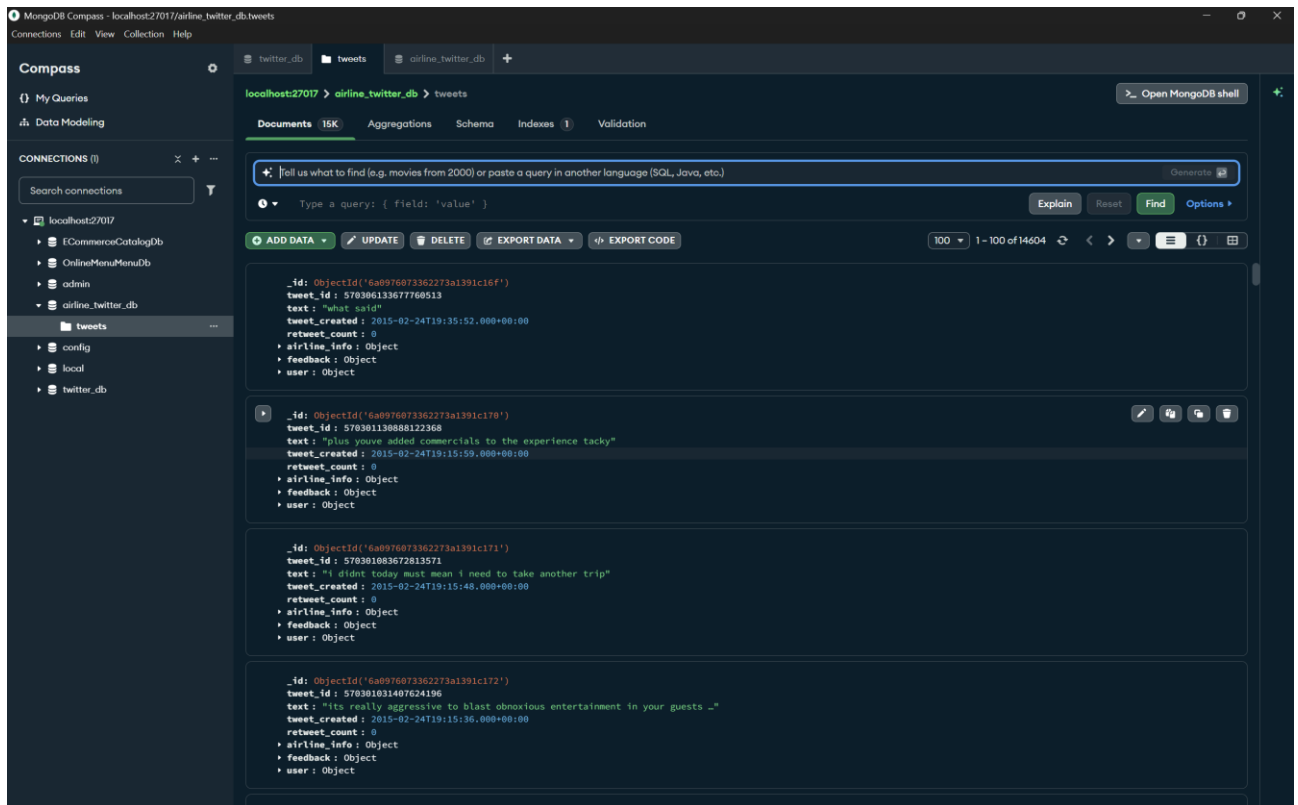
    records = []

    for _, row in df_cleaned.iterrows():
        doc = {
            "tweet_id": row.get("tweet_id"),
            "text": row.get("text"),
            "tweet_created": row.get("tweet_created"),
            "retweet_count": row.get("retweet_count"),
            "airline_info": {
                "name": row.get("airline"),
                "sentiment": row.get("airline_sentiment"),
                "confidence":
row.get("airline_sentiment_confidence")
            },
            "feedback": {
```

```
"negative_reason": row.get("negative_reason")
},
"user": {
    "name": row.get("name"),
    "timezone": row.get("user_timezone")
}
}
records.append(doc)

result = self.collection.insert_many(records)
return len(result.inserted_ids)
```

Sonuç:



5.4 Sorgular ve Veri Tabanı İşlemleri:

Proje yönergesinde belirtildiği gibi sorgular ve işlemler bu bölümde ele alınacaktır.

5.4.1 Find() Metodu Sorguları:

Sorgu 1: Find() ile Koşullu Filtreleme, Projeksiyon ve Limit Optimizasyonu:

Bu sorgunun temel amacı, MongoDB'nin belge yönelimli yapısı üzerinde iç içe dokümanlara erişim yeteneğini ve veri okuma optimizasyonunu kanıtlamaktır.

Sorgu mantığı kapsamında, havayolu şirketi 'United' olan ve duygu durumu 'negative' olarak etiketlenmiş olan kayıtlar filtrelenmiştir. Yalnızca istediğimiz alanları getirmek adına ve sorgu maliyetini düşürmek adına 'projection' tekniği kullanılmıştır. Bu sayede default olarak gelen '_id' alanı getirilmemiş. Yalnızca 'text', 'sentiment' ve 'user' bilgisi getirilmiştir. Son olarak 'limit()' kullanarak yalnızca 5 kayıt getirilmiştir. Konsol üzerinde verilerin düzenli formatta gözükmesi adına ise 'pprint' kullanılmıştır.

Kod:

```
def get_negative_united_tweets(self, limit: int = 10) -> None:
    query = {
        "airline_info.name": "United",
        "airline_info.sentiment": "negative"
    }

    # id gelmesin text, sentiment ve user gelsin
    projection = {
        "_id": 0,
        "text": 1,
        "airline_info.sentiment": 1,
        "user": 1,
    }

    cursor = self.collection.find(query, projection).limit(limit)

    for doc in cursor:
        pprint.pprint(doc)
```

Çıktı:

```

PS C:\Users\ONUR\Desktop\big-data-project> py main.py

--- SORGU 1: Temel find() - Filtreleme ve Projeksiyon ---
Açıklama: United Airlines için negatif duygu durumu içeren tweetler.

{'airline_info': {'sentiment': 'negative'},
 'text': 'still no refund or word via dm please resolve this issue as your '
        'cancelled flight was useless to my assistants trip',
 'user': {'name': 'CoralReefer420', 'timezone': 'Alaska'}}
{'airline_info': {'sentiment': 'negative'},
 'text': 'delayed due to lack of crew and now delayed again because theres a '
        'long line for deicing still need to improve service',
 'user': {'name': 'lsalazar11', 'timezone': 'Mountain Time (US & Canada)'}}
{'airline_info': {'sentiment': 'negative'},
 'text': 'your erior expess connections are hugely popular now if only we '
        'could have an eriewr hop',
 'user': {'name': 'samidip', 'timezone': 'Eastern Time (US & Canada)'}}
{'airline_info': {'sentiment': 'negative'},
 'text': 'you think you boarded flight au1066 too early i think so',
 'user': {'name': 'andycheco', 'timezone': 'Eastern Time (US & Canada)'}}
{'airline_info': {'sentiment': 'negative'},
 'text': 'gate agent hooked me up with alternate flights if you have a way to '
        'prevent the constant issues that would rock',
 'user': {'name': 'slandail', 'timezone': 'Eastern Time (US & Canada)'}}
PS C:\Users\ONUR\Desktop\big-data-project>

```

Sorgu 2: Find() ile Büyüklük/Küçüklük Operatörleri ile Mantıksal Filtreleme

Büyük veri setlerinde istatistiksel aykırılıkları (outliers) veya belirli eşik değerlerini aşan kayıtları tespit etmek için karşılaştırma operatörlerinin kullanımı zorunludur. Bu sorguda, etki alanı geniş olan (viral) olumlu geri bildirimleri analiz etmek amacıyla, MongoDB'nin Büyüktür (Greater Than - \$gt) operatörü kullanılmıştır. Mantıksal AND bağlacı varsayılan olarak işletilerek, retweet_count değeri 0'dan büyük olan ve eş zamanlı olarak airline_info.sentiment değeri "positive" olan kayıtlar tespit edilmiştir.

Kod:

```

def get_retweeted_positive_tweets(self, limit: int = 10) -> None:
    query = {
        "retweet_count": {"$gt": 0},
        "airline_info.sentiment": "positive"
    }
    projection = {
        "_id": 0,
        "text": 1,
        "retweet_count": 1,
        "airline_info.sentiment": 1,
        "user": 1
    }
    cursor = self.collection.find(query, projection).limit(limit)
    for doc in cursor:
        pprint.pprint(doc)

```


Çıktı:

```
--- SORGU 2: Mantıksal Karşılaştırma ($gt) ---
Açıklama: En az 1 kez retweet edilmiş pozitif tweetler.

{'airline_info': {'sentiment': 'positive'},
 'retweet_count': 1,
 'text': 'you rock for making it so i can watch on my flight',
 'user': {'name': 'lisaptv', 'timezone': 'Mountain Time (US & Canada)'}}
{'airline_info': {'sentiment': 'positive'},
 'retweet_count': 2,
 'text': 'always have it together youre welcome rt youre so welcome',
 'user': {'name': 'jessicajaymes', 'timezone': 'Pacific Time (US & Canada)'}}
{'airline_info': {'sentiment': 'positive'},
 'retweet_count': 1,
 'text': 'thanks for gate checking my baggage on your full flight dfwlax 883 '
        'and giving me early boarding too',
 'user': {'name': 'djchuang', 'timezone': 'Pacific Time (US & Canada)'}}
{'airline_info': {'sentiment': 'positive'},
 'retweet_count': 1,
 'text': 'nice rt the man of steel might be faster but we have wifi just '
        'saying',
 'user': {'name': 'miaerolinea', 'timezone': 'Caracas'}}
{'airline_info': {'sentiment': 'positive'},
 'retweet_count': 4,
 'text': 'flying lax to sfo and after looking at the awesome movie lineup i '
        'actually wish i was on a long haul',
 'user': {'name': 'BBahreman', 'timezone': 'Pacific Time (US & Canada)'}}
```

Sorgu 3: Find() ile Veri Sıralama ve Sayfalama Mimarisi:

Milyonlarca satırlık veri tabanlarında sorgu sonuçlarının tamamını tek seferde istemciye yüklemek, bellek taşmalarına ve ağ darboğazlarına yol açmaktadır. Bu sorunun önüne geçmek için Sayfalama mekanizmaları kullanılır. Bu sorguda, sistemdeki en yüksek etkileşimi alan tweetler 'sort("retweet_count", -1)' metodu ile azalan şekilde sıralanmıştır. Ardından, bir web arayüzünde "2. sayfaya geçiş" senaryosu simüle edilerek 'skip(1)' metodu ile en yüksek değerli ilk kayıt atlanmıştır, limit(5) ile takip eden kayıtlar başarılı bir şekilde belleğe alınmıştır.

Kod:

```
def get_top_retweeted_with_pagination(self, skip: int= 1, limit:
int = 10) -> None:
    projection = {
        "_id": 0,
        "text": 1,
        "retweet_count": 1,
        "airline_info.name": 1
    }

    cursor = (self.collection.find({}, projection)
              .sort("retweet_count", -1)
              .skip(skip))
```

```

        .limit(limit))

for doc in cursor:
    pprint.pprint(doc)

```

Çıktı:

```

--- SORGU 3: Sıralama ve Sayfalama (Sort, Skip, Limit) ---
Açıklama: En yüksek retweet alanlardan ilk 1 kaydı atla, sonraki 5 kaydı getir.

{'airline_info': {'name': 'US Airways'},
 'retweet_count': 32,
 'text': 'of course never again tho thanks for tweetin ur concern but not doin '
        'anythin to fix what happened ill choose wiser next time'}
{'airline_info': {'name': 'Delta'},
 'retweet_count': 31,
 'text': 'stop usingthisword if youre a company rt our fleets on fleek'}
{'airline_info': {'name': 'US Airways'},
 'retweet_count': 28,
 'text': 'with this livery back in the day'}
{'airline_info': {'name': 'Delta'},
 'retweet_count': 22,
 'text': 'can you not rt our fleets on fleek'}
{'airline_info': {'name': 'Southwest'},
 'retweet_count': 22,
 'text': 'beautiful day in seattle'}
PS C:\Users\ONUR\Desktop\big-data-project> 

```

Sorgu 4: Find() ile Dizi/Liste İçi Arama ve Kapsam Filtreleme (\$in):

Büyük veri setlerinde aynınahtar üzerinden birden fazla değeri sorgulamak için arka arkaya ‘OR’ blokları yazmak kod okunabilirliğini düşüreceğinden bunu çözmek için MongoDB ‘\$in’ operatörü kullanılmıştır. İlgili sorguda yalnızca “Delta” ve “US Airways” şirketleriyle ilgili ve ‘confidence’ skoru 0.9’un üzerinde olan kayıtlar filtrelenmiştir.

Kod:

```

def get_high_confidence_specific_airlines(self, limit: int = 10) -
> None:
    query = {
        "airline_info.name": {"$in": ["Delta", "US Airways"]},
        "airline_info.confidence": {"$gt": 0.9}
    }
    projection = {
        "_id": 0,
        "airline_info.name": 1,
        "airline_info.confidence": 1,
        "text": 1
    }

```

```

    }

    cursor = self.collection.find(query, projection).limit(limit)
    for doc in cursor:
        pprint.pprint(doc)

```

Çıktı:

```

--- SORGU 4: Liste İçi Arama ($in) ---
Açıklama: Delta veya US Airways için güven skoru > 0.9 olan tweetler.

{'airline_info': {'confidence': 1.0, 'name': 'Delta'},
 'text': 'flight 1041 to savannah ga'}
{'airline_info': {'confidence': 1.0, 'name': 'Delta'},
 'text': 'they werent on any flight they just came late flight your jetblue '
        'employee just informed us'}
{'airline_info': {'confidence': 1.0, 'name': 'Delta'},
 'text': 'everyone is here but our pilots are no where to be found and my last '
        'flight the plane was dirty that i had to clean my area amp seat'}
{'airline_info': {'confidence': 1.0, 'name': 'Delta'},
 'text': 'hey guys why did my last flight earn me 0 pts'}
{'airline_info': {'confidence': 1.0, 'name': 'Delta'},
 'text': '2 aisles of empty seats and we cant move bc we didnt pay'}

```

Sorgu 5: Find() ile Mantıksal VEYA (\$or) Operatörü:

Farklı niteliklere sahip kayıtların tek bir sorguda birleştirilmesi gereken durumlarda mantıksal '\$or' operatörü kullanılmaktadır. Bu sorguda, sosyal medya etkileşiminin yüksekliğini ölçmek amacıyla hibrit bir yapı kurulmuştur. Tweetin retweet sayısının 5'ten büyük olması (\$gt: 5) veya duygu durumunun doğrudan pozitif ("positive") olması koşullarından en az birini sağlayan kayıtlar getirilmiştir.

Kod:

```

def get_highly_retweeted_or_positive_tweets(self, limit:int = 10)
-> None:

    query = {
        "$or": [
            {"retweet_count": {"$gt":5}},
            {"airline_info.sentiment": "positive"}
        ]
    }
    projection = {
        "_id": 0,
        "text": 1,
        "retweet_count": 1,

```

```

        "airline_info.sentiment": 1
    }
    cursor = self.collection.find(query, projection).limit(limit)
    for doc in cursor:
        pprint.pprint(doc)

```

Çıktı:

```

--- SORGU 5: Mantıksal VEYA ($or) ---
Açıklama: Retweet sayısı > 5 VEYA duygu durumu 'positive' olan tweetler.

{'airline_info': {'sentiment': 'positive'},
 'retweet_count': 0,
 'text': 'plus youve added commercials to the experience tacky'}
{'airline_info': {'sentiment': 'positive'},
 'retweet_count': 0,
 'text': 'yes nearly every time i fly vx this ear worm wont go away'}
{'airline_info': {'sentiment': 'positive'},
 'retweet_count': 0,
 'text': 'well i didntbut now i do d'}
{'airline_info': {'sentiment': 'positive'},
 'retweet_count': 0,
 'text': 'it was amazing and arrived an hour early youre too good to me'}
{'airline_info': {'sentiment': 'positive'},
 'retweet_count': 0,
 'text': 'i lt3 pretty graphics so much better than minimal iconography d'}
PS C:\Users\ONUR\Desktop\big-data-project> █

```

5.4.2 Aggregation Pipeline İşlemleri:

Sorgu 1: Kümele ve Aritmetik Özetleme:

Sınıflandırma modelinin temel güvenilirliğini ölçmek amacıyla, havayolu şirketlerine verilen ‘confidence’ skorları analiz edilmiştir. Aggregation hattının ilk aşamasında ‘\$group’ operatörü ile kayıtlar şirket isimlerine göre kümelenmiş ve her bir kümenin ortalaması ‘\$avg’ operatörü ile hesaplanmıştır. İkinci aşamada ise ‘\$sort’ operatörü kullanılarak sonuçlar büyükten küçüğe sıralanmış ve en güvenilir etiketlenen şirketten en aza doğru giden bir çıktı üretilmiştir.

Kod:

```

def get_avg_confidence_by_airline(self) -> None:
    pipeline = [
        {"$group": {
            "_id": "$airline_info.name",
            "avg_confidence": {"$avg": "$airline_info.confidence"}
        }},
        {"$sort": {"avg_confidence": -1}}
    ]
    cursor = self.collection.aggregate(pipeline)

```

```
for doc in cursor:
    pprint.pprint(doc)
```

Çıktı:

```
--- SORGU 1: Aggregation Pipeline ($group, $avg, $sort) ---
Açıklama: Havayolu şirketlerinin ortalama duygu güven skorları (Azalan Sırayla).
{'_id': 'US Airways', 'avg_confidence': 0.9215784414692757}
{'_id': 'American', 'avg_confidence': 0.9162592728608153}
{'_id': 'United', 'avg_confidence': 0.9008776818419676}
{'_id': 'Southwest', 'avg_confidence': 0.8865159504132231}
{'_id': 'Virgin America', 'avg_confidence': 0.8760861111111111}
{'_id': 'Delta', 'avg_confidence': 0.8698782628262827}
```

Sorgu 2: Çok Katmanlı Veri İndirgeme ve Kök Neden Analizi:

Bir havayolu şirketinin müşteri memnuniyetsizliği kök nedenlerini bulmak için bir pipeline oluşturulmuştur. İlk aşamada '\$match' operatörü ile sadece "US Airways" ve "negative" olan kayıtlar pipeline içine alınmıştır. İkinci aşamada '\$group' ile şikayet nedenleri kümelenmiş ve her birinin frekansları sayılmıştır. Son aşamada ise '\$sort' ile sıralanmış ve '\$limit' ile en en kritik 5 tanesi elde edilmiştir.

Kod:

```
def get_top_complaints_for_us_airways(self, limit: int = 10) ->
None:
    pipeline = [
        {"$match": {
            "airline_info.name": "US Airways",
            "airline_info.sentiment": "negative"
        }},
        {"$group": {
            "_id": "$feedback.negative_reason",
            "count": {"$sum": 1}
        }},
        {"$sort": {"count": -1}},
        {"$limit": limit}
    ]
    cursor = self.collection.aggregate(pipeline)
    for doc in cursor:
```

```
pprint.pprint(doc)
```

Çıktı:

```
--- SORGU 2: Zincirleme Aggregation ($match, $group, $sort, $limit) ---  
Açıklama: US Airways için en sık karşılaşılan ilk 5 şikayet nedeni.  
  
{'_id': 'Customer Service Issue', 'count': 811}  
{'_id': 'Late Flight', 'count': 453}  
{'_id': 'Can't Tell', 'count': 246}  
{'_id': 'Cancelled Flight', 'count': 189}  
{'_id': 'Lost Luggage', 'count': 154}  
PS C:\Users\ONUR\Desktop\big-data-project> []
```

Sorgu 3: Veri Şekillendirme ve Lokalizasyon:

MongoDB'den çekilen ham verilerin çıktıya aktarılmadan önce isimlendirilmesi ve formatlanması kritik bir adımdır. Bu sorguda, '\$group' operatörü ile veri setindeki genel duygu durum (sentiment) dağılımı hesaplanmış, ardından pipeline'ın son aşamasına '\$project' operatörü eklenmiştir. '\$project' sayesinde veritabanındaki varsayılan İngilizce alan isimleri (_id ve total), sunum standartlarına uygun olarak Türkçe ("Duygu_Durumu" ve "Toplam_Tweet_Sayisi") şeklinde yeniden yapılandırılmıştır.

Kod:

```
def get_sentiment_distribution(self) -> None:  
    pipeline = [  
        {"$group": {  
            "_id": "$airline_info.sentiment",  
            "total": {"$sum": 1}  
        }},  
        {"$project": {  
            "_id": 0,  
            "Duygu_Durumu": "$_id",  
            "Toplam_Tweet_Sayisi": "$total"  
        }}  
    ]  
    cursor = self.collection.aggregate(pipeline)  
    for doc in cursor:  
        pprint.pprint(doc)
```

Çıktı:

```
--- SORGU 3: Projeksiyon ile İsimlendirme ($project) ---  
Açıklama: Duygu dağılımının Türkçe isimlendirilerek raporlanması.  
{'Duygu_Durumu': 'neutral', 'Toplam_Tweet_Sayisi': 3091}  
{'Duygu_Durumu': 'negative', 'Toplam_Tweet_Sayisi': 9159}  
{'Duygu_Durumu': 'positive', 'Toplam_Tweet_Sayisi': 2354}
```

Sorgu 4: Sürekli Verilerin Sepetlenmesi ve Histogram Analizi:

Makine öğrenmesi modelleri eğitilmeden önce sürekli sayısal verilerin belirli aralıklara ayrılarak analiz edilmesi veri kalitesini artırmaktadır. Bu sorguda MongoDB'nin ileri düzey '\$bucket' operatörü kullanılarak, confidence skorları 3 farklı aralığa bölünmüştür. Ayrıca '\$addToSet' dizi operatörü kullanılarak, bu güven aralıklarında değerlendirilen havayolu şirketleri tekrarsız bir liste olarak rapora dahil edilmiştir.

Kod:

```
def get_confidence_score_histogram(self) -> None:  
    pipeline = [  
        {"$bucket": {  
            "groupBy": "$airline_info.confidence",  
            "boundaries": [0.0, 0.5, 0.8, 1.01],  
            "default": "NoN",  
            "output": {  
                "tweet_sayisi": {"$sum": 1},  
                "etkilenen_sirketler": {"$addToSet":  
"$airline_info.name"}  
            }  
        }}  
    ]  
    cursor = self.collection.aggregate(pipeline)  
    for doc in cursor:  
        pprint.pprint(doc)
```

Çıktı:

```

--- SORGU 4: Histogram Oluşturma ($bucket) ---
Açıklama: Etiket güven skorlarının belirli aralıklara (sepetlere) bölünmesi.

{'_id': 0.0,
 'etkilenen_sirketler': ['Southwest',
                        'Virgin America',
                        'United',
                        'Delta',
                        'US Airways',
                        'American'],
 'tweet_sayisi': 236}
{'_id': 0.5,
 'etkilenen_sirketler': ['United',
                        'Virgin America',
                        'Southwest',
                        'Delta',
                        'American',
                        'US Airways'],
 'tweet_sayisi': 3945}
{'_id': 0.8,
 'etkilenen_sirketler': ['Southwest',
                        'Virgin America',
                        'United',
                        'Delta',
                        'American',
                        'US Airways'],
 'tweet_sayisi': 10423}

```

Sorgu 5: Tekil Sorguda Çoklu İş Zekası Raporu:

Yüksek trafiğe sahip büyük veri sistemlerinde aynı veri seti üzerinden farklı analizler yapmak için veri tabanına art arda sorgular atmak ciddi performans darboğazları yaratır. Bu mimari sorunu çözmek adına MongoDB'nin '\$facet' operatörü kullanılmıştır. Bu operatör sayesinde veri tabanına tek bir istek gönderilmiş ve pipeline iki paralel dala ayrılarak; hem "En çok pozitif tweet alan 3 şirket" alt raporu hem de "En sık karşılaşılan 3 müşteri şikâyeti" alt raporu aynı anda, tek bir JSON dokümanı içinde başarıyla elde edilmiştir.

Kod:

```

def get_comprehensive_summary_report(self) -> None:
    pipeline = [
        {"$facet": {
            "en_ iyi_sirketler": [
                {"$match": {"airline_info.sentiment":
"positive"}},
                {"$group": {"_id": "$airline_info.name",
"pozitif_sayisi": {"$sum": 1}}},
                {"$sort": {"pozitif_sayisi": -1}},
                {"$limit": 5}
            ],
            "en_cok_sikayetler": [

```



```

        {"$match": {"airline_info.sentiment":
"negative"}},
        {"$group": {"_id": "$feedback.negative_reason"
,"sikayet_sayisi": {"$sum": 1}}},
        {"$sort": {"sikayet_sayisi": -1}},
        {"$limit": 5}

    ]

    }}

]
cursor = self.collection.aggregate(pipeline)
for doc in cursor:
    pprint.pprint(doc)

```

Çıktı:

```

--- SORGU 5: Çoklu Raporlama ($facet) ---
Açıklama: Tek sorguda 'En İyi Şirketler' ve 'En Sık Şikayetler' raporları.

{'en_cok_sikayetler': [{'_id': 'Customer Service Issue',
'sikayet_sayisi': 2984},
{'_id': 'Late Flight', 'sikayet_sayisi': 1660},
{'_id': 'Can't Tell', 'sikayet_sayisi': 1190},
{'_id': 'Cancelled Flight', 'sikayet_sayisi': 843},
{'_id': 'Lost Luggage', 'sikayet_sayisi': 721}],
'en_ iyi_sirketler': [{'_id': 'Southwest', 'pozitif_sayisi': 570},
{'_id': 'Delta', 'pozitif_sayisi': 544},
{'_id': 'United', 'pozitif_sayisi': 492},
{'_id': 'American', 'pozitif_sayisi': 327},
{'_id': 'US Airways', 'pozitif_sayisi': 269}]]
PS C:\Users\ONUR\Desktop\big-data-project>

```

5.4.3 Index İşlemleri:

Sorgu 1: B-Tree Indexi ve Yürütme İstatistikleri:

Filtreleme operasyonlarında sistemin davranışsal analizi için 'airline_info.name: "Delta"' sorgusu üzerinden yürütme planı oluşturulmuştur.

- **Index Öncesi Yapı:** Herhangi bir index tanımı bulunmazken veri tabanı, aranan 2222 adet kayda ulaşmak için tüm veriyi baştan sonra taramak zorunda kalmıştır. Bu işlem 7 ms sürmüştür.
- **Index Sonrası Yapı:** 'airline_info.name' alanı üzerinden B-Tree indeksi oluşturulduktan sonra aynı sorguda taranan veri sayısı aranan veri sayısı ile aynı olmuştur (2222 adet). Ancak bu işlem 9 ms sürmüştür.

Buradaki süre farkı şundan kaynaklanmıştır:

Küçük veri setleri belleğe tamamen sığıldığı için, indeks ağacını (B-Tree) taramanın yarattığı ek yönlendirme maliyeti süreyi 7 ms'den 9 ms'ye çıkarmıştır. Ancak veri boyutu milyonlara ulaştığında, logaritmik çalışan indeks mimarisi doğrusal taramanın sistemi kilitlemesini önleyecek ve asıl performans avantajını o zaman ortaya koyacaktır.

Kod:

```
def test_index_performance(self) -> None:
    self.collection.drop_indexes()
    query = {"airline_info.name": "Delta"}
    # index yokken performans
    plan_before = self.collection.find(query).explain(
["executionStats"]
    docs_before = plan_before.get("totalDocsExamined", 0)
    time_before = plan_before.get("executionTimeMillis", 0)
    print(f"İndeks ÖNCESİ -> Taranan Doküman: {docs_before}, Süre:
{time_before} ms")

    # index oluşturma
    self.collection.create_index([("airline_info.name", 1)])
    print("\n[*] 'airline_info.name' alanı üzerine B-Tree indeksi
oluşturuldu.\n")

    # İndeks varken sorgu performansı
    plan_after = self.collection.find(query).explain()["executionStats"]
    docs_after = plan_after.get("totalDocsExamined", 0)
    time_after = plan_after.get("executionTimeMillis", 0)
    print(f"İndeks SONRASI -> Taranan Doküman: {docs_after}, Süre:
{time_after} ms")
```

Çıktı:

```
PS C:\Users\ONUR\Desktop\big-data-project> py main.py

--- SORGU 1: İndeksleme ve Performans Testi (explain) ---
Açıklama: İndeks öncesi ve sonrası taranan doküman sayılarının kıyaslanması.

İndeks ÖNCESİ -> Taranan Doküman: 14604, Süre: 7 ms

[*] 'airline_info.name' alanı üzerine B-Tree indeksi oluşturuldu.

İndeks SONRASI -> Taranan Doküman: 2222, Süre: 9 ms
```

Sorgu 2: Full-Text Index ve Skorlama:

Yapılandırılmamış serbest metinlerde anlamsal aramalar yapmak için geleneksel ve maliyetli '\$regex' sorguları yerine text sütunu üzerine Full-Text Index inşa edilmiştir. Bu özel indeks mimarisi üzerinden '\$text ve \$search' operatörleri tetiklenerek "baggage" (bagaj) ve "lost" (kayıp) kelimeleri taranmıştır. Klasik filtreleme mantığından farklı olarak bu sorgu, dokümanları sadece bulmakla kalmamış; kelimelerin metin içerisindeki yoğunluğuna, konumuna ve önemine göre her belgeye ağırlıklı bir Alaka Skoru atamıştır.

Kod:

```
def search_with_text_index(self, limit: int = 10) -> None:
    # Text Index Oluşturma
    self.collection.create_index([("text", "text")])
    print("[*] 'text' alanı üzerine Full-Text indeksi
oluşturuldu.\n")
    query = {
        "$text": {"$search": "baggage lost"}
    }
    projection = {
        "_id": 0,
        "airline_info.name": 1,
        "text": 1,
        "alaka_skoru": {"$meta": "textScore"}
    }
    cursor = (self.collection.find(query, projection)
               .sort([("alaka_skoru", {"$meta": "textScore"})])
               .limit(limit))
    for doc in cursor:
        pprint.pprint(doc)
```

Çıktı:

```

--- SORGU 2: Full-Text İndeks ve Kelime Araması ($text) ---
Açıklama: 'baggage' veya 'lost' kelimelerini içeren tweetlerin Text Index ile aranması.

[*] 'text' alanı üzerine Full-Text indeksi oluşturuldu.

{'airline_info': {'name': 'Delta'},
 'alaka_skoru': 1.1428571428571428,
 'text': 'second incident of lost baggage i sent you a dm thoughts'}
{'airline_info': {'name': 'American'},
 'alaka_skoru': 1.125,
 'text': 'hellomy baggage has been lost in another city can anyone help me '
        'with this'}
{'airline_info': {'name': 'Southwest'},
 'alaka_skoru': 1.125,
 'text': 'how can customers get in touch with you internationally from mexico '
        'for lost baggage'}
{'airline_info': {'name': 'Southwest'},
 'alaka_skoru': 1.125,
 'text': 'do you know that your lost baggage portal doesnt work on mobile its '
        'a 404'}
{'airline_info': {'name': 'US Airways'},
 'alaka_skoru': 1.1111111111111112,
 'text': 'they lost my bag baggage handlers broke open my bag and stole my '
        'camera'}
PS C:\Users\ONUR\Desktop\big-data-project>

```

5.4.4 Update İşlemleri:

Sorgu 1: Tekli Doküman Genişletme:

Klasik SQL tabanlı sistemlerde mevcut bir tabloya yeni bir sütun eklemek ALTER TABLE komutunu ve tüm tablonun kilitlenmesini gerektirirken, MongoDB'de dokümanlar birbirinden bağımsız yapıdadır. Bu durumu kanıtlamak amacıyla 'updateOne()' metodu kullanılarak spesifik bir şikâyet kaydı hedeflenmiştir. \$set operatörü ile bu dokümanın içerisine daha önce şemada hiç var olmayan 'is_reviewed, reviewer_note ve action_taken' gibi yepyeni alanlar anında enjekte edilmiştir.

Kod:

```

def review_single_complaint(self) -> None:
    query = {
        "airline_info.name": "Virgin America",
        "airline_info.sentiment": "negative"
    }
    update_operation = {
        "$set": {
            "is_reviewed": True,
            "reviewer_note": "Müşteri hizmetleri departmanına
iletildi.",
            "action_taken": "Pending"
        }
    }
    result = self.collection.update_one(query, update_operation)

```

```

print(f"Eşleşen Doküman Sayısı: {result.matched_count}")
print(f"Güncellenen Doküman Sayısı: {result.modified_count}")
if result.modified_count > 0:
    print("\n[+] Güncellenen Dokümanın Son Hali:")
    updated_doc = self.collection.find_one(query)
    pprint.pprint(updated_doc)

```

Çıktı:

```

PS C:\Users\ONUR\Desktop\big-data-project> py main.py

--- SORGU 1: Tekil Doküman Güncelleme ($set) ---
Açıklama: Spesifik bir şikayete 'is_reviewed' bayrağı eklenmesi.

Eşleşen Doküman Sayısı: 1
Güncellenen Doküman Sayısı: 1

[+] Güncellenen Dokümanın Son Hali:
{'_id': ObjectId('6a0976073362273a1391c172'),
 'action_taken': 'Pending',
 'airline_info': {'confidence': 1.0,
                  'name': 'Virgin America',
                  'sentiment': 'negative'},
 'feedback': {'negative_reason': 'Bad Flight'},
 'is_reviewed': True,
 'retweet_count': 0,
 'reviewer_note': 'Müşteri hizmetleri departmanına iletili.',
 'text': 'its really aggressive to blast obnoxious entertainment in your '
         'guests faces amp they have little recourse',
 'tweet_created': datetime.datetime(2015, 2, 24, 19, 15, 36),
 'tweet_id': 570301031407624196,
 'user': {'name': 'jnardino', 'timezone': 'Pacific Time (US & Canada)'}}

```

Sorgu 2: Toplu Veri Güncelleme:

Bu metotta, pozitif duygu durumuna sahip tüm tweetlerin 'retweet_count' değerlerini artırmak için 'updateMany()' fonksiyonu ve MongoDB'nin atomik matematik operatörü olan '\$inc' kullanılmıştır. Sorgu sonucunda, eşleşen binlerce dokümanın retweet sayısı tek bir I/O işlemiyle ve milisaniyeler içinde doğrudan disk üzerinde matematiksel olarak artırılmıştır.

Kod:

```

def boost_positive_tweet_retweets(self) -> None:
    query = {
        "airline_info.sentiment": "positive"
    }
    update_operation = {
        "$inc": {
            "retweet_count": 5
        }
    }

```

```

    }
    result = self.collection.update_many(query, update_operation)
    print(f"Eşleşen Doküman Sayısı: {result.matched_count}")
    print(f"Güncellenen Doküman Sayısı: {result.modified_count}")
    if result.modified_count > 0:
        print("\n[+] Örnek Güncellenmiş Doküman (Retweet sayısı kontrolü):")
        sample_doc = self.collection.find_one(query)
        pprint.pprint(sample_doc)

```

Çıktı:

```

--- SORGU 2: Toplu Doküman Güncelleme ($inc) ---
Açıklama: Pozitif tweetlerin retweet sayısının topluca artırılması.

Eşleşen Doküman Sayısı: 2354
Güncellenen Doküman Sayısı: 2354

[+] Örnek Güncellenmiş Doküman (Retweet sayısı kontrolü):
{'_id': ObjectId('6a0976073362273a1391c170'),
 'airline_info': {'confidence': 0.3486,
                  'name': 'Virgin America',
                  'sentiment': 'positive'},
 'feedback': {'negative_reason': 'Unknown'},
 'retweet_count': 5,
 'text': 'plus youve added commercials to the experience tacky',
 'tweet_created': datetime.datetime(2015, 2, 24, 19, 15, 59),
 'tweet_id': 570301130888122368,
 'user': {'name': 'jnardino', 'timezone': 'Pacific Time (US & Canada)'}}
PS C:\Users\ONUR\Desktop\big-data-project> 

```

5.4.5 Delete İşlemleri:

Sorgu 1: Toplu Veri Silme:

Bu kısımda, etiketleyicilerin kararsız kaldığı ve makine öğrenmesi modelini yanıltma (bias) riski yüksek olan kayıtlar tespit edilmiştir. Confidence skoru %50'nin (0.5) altında kalan zayıf etiketli veriler, MongoDB'nin 'deleteMany()' operatörü ile koleksiyondan toplu ve kalıcı olarak temizlenmiştir.

Kod:

```

def delete_low_confidence_tweets(self) -> None:
    query = {
        "airline_info.confidence": {"$lt": 0.50}
    }
    result = self.collection.delete_many(query)
    print(f"Filtreye Uyan ve Kalıcı Olarak Silinen Doküman Sayısı: {result.deleted_count}")

```

Çıktı:

```
PS C:\Users\ONUR\Desktop\big-data-project> py main.py
--- SORGU 1: Toplu Kayıt Silme (deleteMany) ---
Açıklama: Etiket güven skoru < 0.50 olan zayıf verilerin temizlenmesi.
Filtreye Uyan ve Kalıcı Olarak Silinen Doküman Sayısı: 0
```

Sorgu 2: Tekli Veri Silme:

Büyük veri operasyonlarında, sisteme sızan spesifik bir anomaliyi veya kirliliği temizlerken, benzer niteliklere sahip diğer geçerli verilerin zarar görmesini engellemek kritik bir veri güvenliği prensibidir. Bu doğrultuda, toplu silme komutlarının yaratabileceği veri kaybı riskini izole etmek amacıyla MongoDB'nin 'deleteOne()' metodu kullanılarak "nokta atışı" bir temizlik işlemi simüle edilmiştir.

Kod:

```
def delete_anomalous_tweet(self) -> None:
    query = {
        "airline_info.sentiment": "neutral"
    }
    result = self.collection.delete_one(query)
    print(f"Anomali Olarak Tespit Edilip Silinen Tekil Doküman Sayısı: {result.deleted_count}")
```

Çıktı:

```
--- SORGU 2: Tekil Kayıt Silme (deleteOne) ---
Açıklama: Anomali yaratan spesifik tek bir kaydın güvenli silinmesi.
Anomali Olarak Tespit Edilip Silinen Tekil Doküman Sayısı: 1
PS C:\Users\ONUR\Desktop\big-data-project>
```

5.4.6 İleri Düzey Sorgular:

Sorgu 1: İç İçe Sorgu:

Bu sorgunun amacı, ilişkisel veritabanlarındaki JOIN operasyonlarının yarattığı performans maliyetlerinden kaçınarak, iç içe JSON mimarisi üzerinde coğrafi konum ve duygu durum tabanlı bir çapraz filtreleme yapmaktır. Operasyon kapsamında, MongoDB'nin nokta notasyonu kullanılarak 'user.timezone' alt alanında "Eastern Time" zaman dilimi aranmış ve eş zamanlı olarak 'airline_info.sentiment' alanı üzerinden negatif geri bildirimler filtrelenmiştir. Çekilen sonuçlar, yalnızca analize konu olacak alt objeleri belleğe alacak şekilde projection işleminden geçirilerek ağ I/O yükü minimize edilmiştir.

Kod:

```

def get_eastern_time_negative_users(self, limit: int = 10) ->
None:
    query = {
        "user.timezone": {"$regex": "Eastern Time", "$options":
        "i"},
        "airline_info.sentiment": "negative"
    }
    projection = {
        "_id": 0,
        "user.name": 1,
        "user.timezone": 1,
        "feedback.negative_reason": 1
    }
    cursor = self.collection.find(query, projection).limit(limit)
    for doc in cursor:
        pprint.pprint(doc)

```

Çıktı:

```

--- SORGU 1: İç İçe Doküman (Nested) Araması ---
Açıklama: 'Eastern Time' bölgesindeki kullanıcıların şikayet nedenleri.

{'feedback': {'negative_reason': 'Bad Flight'},
 'user': {'name': 'heatherovieda', 'timezone': 'Eastern Time (US & Canada)'}}
{'feedback': {'negative_reason': 'Flight Booking Problems'},
 'user': {'name': 'murphicus', 'timezone': 'Eastern Time (US & Canada)'}}
{'feedback': {'negative_reason': 'Customer Service Issue'},
 'user': {'name': 'rachie1126', 'timezone': 'Eastern Time (US & Canada)'}}
{'feedback': {'negative_reason': 'Late Flight'},
 'user': {'name': 'noelduan', 'timezone': 'Eastern Time (US & Canada)'}}
{'feedback': {'negative_reason': 'Lost Luggage'},
 'user': {'name': 'gianagon', 'timezone': 'Eastern Time (US & Canada)'}}

```

Sorgu 2: Regex ile Sorgu:

Bu sorguda, text alanı üzerinde MongoDB'nin '\$regex' operatörü devreye sokulmuştur. Arama esnasında büyük/küçük harf duyarsızlığını sağlamak için 'i' opsiyonu eklenmiş ve mantıksal 'VEYA' bağlacı kullanılarak metninde "baggage", "lost" veya "luggage" kelimelerinden en az birini barındıran bagaj odaklı geri bildirimler veri kümesinden başarıyla izole edilmiştir.

Kod:

```

def search_baggage_issues_regex(self, limit: int = 10) -> None:
    regex_pattern = re.compile(r"baggage|lost|luggage",
re.IGNORECASE)

```



```

query = {
    "text": {"$regex": regex_pattern}
}

projection = {
    "_id": 0,
    "airline_info.name": 1,
    "text": 1
}

cursor = self.collection.find(query, projection).limit(limit)
for doc in cursor:
    pprint.pprint(doc)

```

Çıktı:

```

--- SORGU 2: Regex ile Metin İçi Kelime Araması ($regex) ---
Açıklama: Bagaj kayıp sorunu yaşayan müşteri bildirimleri.

{'airline_info': {'name': 'Virgin America'},
 'text': 'everything was fine until you lost my bag'}
{'airline_info': {'name': 'Virgin America'},
 'text': 'hi i just booked a flight but need to add baggage how can i do this'}
{'airline_info': {'name': 'Virgin America'},
 'text': 'why must a traveler miss a flight to late flight check a bag i '
        'missed my morning appointments and you lost my business'}
{'airline_info': {'name': 'Virgin America'},
 'text': 'your no late flight bag check just lost you my business i missed '
        'flight and am apt three other people on flight had same exp'}
{'airline_info': {'name': 'Virgin America'},
 'text': 'youd think paying an extra 100 bucks rt for luggage might afford you '
        'hiring an extra hand at'}

```

Sorgu 3: Doküman içi Dinamik Veri Hesaplama Sorgusu:

Bu operasyonun temel amacı, müşteri geri bildirimlerinin detay seviyesini ölçerek yalnızca yüksek bağlama sahip kapsamlı şikâyet metinlerini filtrelemektir. İşlem sırasında, NoSQL sistemlerinin dinamik hesaplama yeteneğinden faydalanılmış ve sorgu mekanizmasında '\$expr' operatörü kullanılmıştır. Bu sayede, text alanındaki her bir kaydın karakter uzunluğu veritabanı üzerinde '\$strLenCP' fonksiyonuyla anlık olarak hesaplanmış ve uzunluğu 100 karakteri aşan detaylı negatif dokümanlar getirilmiştir.

Kod:

```

def get_long_complaints(self, limit: int = 10) -> None:
    query = {
        "airline_info.sentiment": "negative",
        "$expr": {"$gt": [{"$strLenCP": "$text"}, 100]}
    }
    projection = {

```

```

        "_id": 0,
        "text": 1,
        "airline_info.name": 1
    }
    cursor = self.collection.find(query, projection).limit(limit)
    for doc in cursor:
        pprint.pprint(doc)

```

Çıktı:

```

--- SORGU 3: Doküman İçi Uzunluk Fonksiyonu ($expr ve $strlenCP) ---
Açıklama: 100 karakterden uzun, detaylandırılmış müşteri şikayetleri.

{'airline_info': {'name': 'Virgin America'},
 'text': 'its really aggressive to blast obnoxious entertainment in your '
        'guests faces amp they have little recourse'}
{'airline_info': {'name': 'Virgin America'},
 'text': 'seriously would pay 30 a flight for seats that didnt have this '
        'playing its really the only bad thing about flying va'}
{'airline_info': {'name': 'Virgin America'},
 'text': 'i flew from nyc to sfo last week and couldnt fully sit in my seat '
        'due to two large gentleman on either side of me help'}
{'airline_info': {'name': 'Virgin America'},
 'text': 'why are your first fares in may over three times more than other '
        'carriers when all seats are available to select'}
{'airline_info': {'name': 'Virgin America'},
 'text': 'you guys messed up my seating i reserved seating with my friends and '
        'you guys gave my seat away i want free internet'}

```

Sorgu 4: Belirli Zaman Aralığındaki Verileri Getirme Sorgusu:

Bu sorgunun amacı, Twitter verilerini zaman ekseninde analiz ederek belirli bir periyottaki uçuş gecikmelerini tarihsel olarak tespit etmektir. Önceki aşamada ISO standartlarında 'datetime' formatına dönüştürülen 'tweet_created' alanı referans alınarak bir zaman serisi filtresi oluşturulmuştur. Bu kapsamda, '\$gte ve \$lt' operatörleri kullanılarak veri seti 22-24 Şubat 2015 tarihleri arasına sınırlandırılmış ve metninde "delay" ifadesi geçen tweetler yeniden eskiye doğru listelenerek gecikme analizinin veritabanı altyapısı hazırlanmıştır.

Kod:

```

def get_delay_tweets_in_time_range(self, limit: int = 10) -> None:
    start_date = datetime(2015, 2, 22)
    end_date = datetime(2015, 2, 25)
    query = {
        "tweet_created": {"$gte": start_date, "$lt": end_date},
        "text": {"$regex": "delay", "$options": "i"}
    }
    projection = {

```

```

        "_id": 0,
        "tweet_created": 1,
        "airline_info.name": 1,
        "text": 1
    }
    cursor = self.collection.find(query,
projection).sort("tweet_created", -1).limit(limit)
    for doc in cursor:
        pprint.pprint(doc)

```

Çıktı:

```

--- SORGU 4 (BONUS): Zaman Serisi (Time-Series) Analizi ---
Açıklama: 22-24 Şubat 2015 arasında yaşanan uçuş gecikmesi (delay) bildirimleri.

{'airline_info': {'name': 'United'},
 'text': 'delayed due to lack of crew and now delayed again because theres a '
        'long line for deicing still need to improve service',
 'tweet_created': datetime.datetime(2015, 2, 24, 19, 39, 25)}
{'airline_info': {'name': 'Southwest'},
 'text': 'my flight was delayed 4 hrs im 5 months pregnant amp supposed to be '
        'caring for my mom whose in surgery now very upset',
 'tweet_created': datetime.datetime(2015, 2, 24, 19, 26, 35)}
{'airline_info': {'name': 'American'},
 'text': 'i ordered it as i always do but on a 9hour flight delayed for 4 '
        'hours it was worse than ever before when you forgot my meal',
 'tweet_created': datetime.datetime(2015, 2, 24, 18, 37, 27)}
{'airline_info': {'name': 'American'},
 'text': 'i ordered it as i always do but on a 9hour flight delayed for 4 '
        'hours it was worse than ever before when you forgot my meal',
 'tweet_created': datetime.datetime(2015, 2, 24, 18, 37, 27)}
{'airline_info': {'name': 'Southwest'},
 'text': 'oh look you just delayed our flight an additional half hour now '
        'arriving at 6 officially five hours after planned',
 'tweet_created': datetime.datetime(2015, 2, 24, 18, 29, 51)}
PS C:\Users\ONUR\Desktop\big-data-project>

```

6. PySpark ile Dağıtık Veri İşleme:

Büyük veri analitiği süreçlerinde veri hacmi büyüdükçe tekil makinelerin bellek ve işlemci kapasiteleri yetersiz kalmakta, dikey ölçekleme sınırlarına ulaşılmaktadır. Bu problemin üstesinden gelmek ve yatayda paralel veri işlemeyi simüle etmek amacıyla projenin bu aşamasında küme tabanlı, bellekte veri işleme mimarisine sahip Apache Spark altyapısı kullanılmıştır. Sistemin Python ekosistemiyle entegrasyonu PySpark kütüphanesi aracılığıyla sağlanmıştır.

6.1. Verinin PySpark'a Yüklenmesi:

Pipeline'ın kesintisiz ve hatasız çalışması amacıyla, Aşama 2'de ön işlemesi tamamlanan veriler, doğrudan veritabanı soketleri üzerinden aktarılacak yerine 'cleaned_tweets.csv'

dosyasına dışa aktarılmıştır. Bu sayede farklı sistemler arası (MongoDB -> JVM) oluşabilecek altyapı darboğazları izole edilmiştir.

İşlemlerin başlatılması için yerel bilgisayardaki tüm işlemci çekirdeklerini (local[*]) paralel birer işçi düğümü olarak yapılandırılan bir SparkSession nesnesi ayağa kaldırılmıştır. Spark'ın yerel Java Sanal Makinesi (JVM) tabanlı optimize dosya okuyucusu tetiklenerek veri seti belleğe dağıtık bir DataFrame yapısında yüklenmiştir. Yükleme esnasında inferSchema parametresi aktif edilerek veritiplerinin (timestamp, double, integer vb.) Spark motoru tarafından otomatik olarak çözümlenmesi ve yapılandırılması sağlanmıştır.

Kod:

```
class SparkManager:
    def __init__(self, app_name: str = "TwitterSparkAnalysis"):
        self.spark = (SparkSession.builder
                      .appName(app_name)
                      .master("local[*]")
                      .getOrCreate())
        self.spark.sparkContext.setLogLevel("WARN")
        self.df = None

    def load_cleaned_csv(self, file_path: str =
"data/processed/cleaned_tweets.csv") -> None:
        print(f"Temizlenmiş CSV verisi dağıtık sisteme yükleniyor:
{file_path}")
        if not os.path.exists(file_path):
            raise FileNotFoundError(f"Dosya bulunamadı: {file_path}.")
        self.df = (self.spark.read
                  .option("header", "true")
                  .option("inferSchema", "true")
                  .csv(file_path))

        print("\nDağıtık Spark DataFrame Şeması (printSchema):")
        self.df.printSchema()
```

Çıktı:

```

PS C:\Users\ONUR\Desktop\big-data-project> py main.py
WARNING: Using incubator modules: jdk.incubator.vector
26/05/18 16:50:16 WARN Shell: Did not find winutils.exe: java.io.FileNotFoundException: java.io.FileNotFoundException: HADOOP_HOME and hadoop.home
Using Spark's default log4j profile: org/apache/spark/log4j2-defaults.properties
Setting default log level to "WARN".
To adjust logging level use sc.setLogLevel(newLevel). For SparkR, use setLogLevel(newLevel).
26/05/18 16:50:16 WARN NativeCodeLoader: Unable to load native-hadoop library for your platform... using builtin-java classes where applicable
Temizlenmiş CSV verisi dağıtık sisteme yükleniyor: data/processed/cleaned_tweets.csv

Dağıtık Spark DataFrame Şeması (printSchema):
root
|-- tweet_id: string (nullable = true)
|-- airline_sentiment: string (nullable = true)
|-- airline_sentiment_confidence: double (nullable = true)
|-- negativereason: string (nullable = true)
|-- negativereason_confidence: double (nullable = true)
|-- airline: string (nullable = true)
|-- name: string (nullable = true)
|-- retweet_count: integer (nullable = true)
|-- text: string (nullable = true)
|-- tweet_created: timestamp (nullable = true)
|-- tweet_location: string (nullable = true)
|-- user_timezone: string (nullable = true)

```

6.2. Dağıtık Veri Dönüşüm Operasyonları (Transformations):

İşlem 1: Koşullu Veri Filtreleme ve Sütun Seçimi:

Bu işlemin amacı, dağıtık bellek üzerindeki büyük veri seti içerisinde iş zekâsı gereksinimlerine uygun anlamsal bir alt küme oluşturarak veri hacmini daraltmaktır. Operasyon kapsamında, DataFrame yapısı üzerinden 'select()' dönüşümü kullanılarak gereksiz sütunlar elenmiş; yalnızca 'airline', 'airline_sentiment' ve 'text' alanları seçilmiştir. Ardından 'filter()' operatörüyle mantıksal bir AND yapısı kurularak sadece "US Airways" şirketine ait "negative" etiketli müşteri şikayetleri getirilmiştir.

Kod:

```

def filter_negative_us_airways(self) -> None:
    if self.df is None:
        print("Hata: Önce veri setini yüklemelisiniz!")
        return
    transformed_df = (self.df
                      .select("airline", "airline_sentiment",
                              "text")
                      .filter((col("airline") == "US Airways") &
                              (col("airline_sentiment") ==
                               "negative"))
                      transformed_df.show(5, truncate=False)

```

Çıktı:

```

--- İŞLEM 1 (Transformation): select() ve filter() ---
Açıklama: US Airways havayoluna ait olumsuz (negative) tweetlerin ayıklanması.

+-----+-----+-----+
|airline |airline_sentiment|text
+-----+-----+-----+
|US Airways|negative        |is there a better time to call my flight is on friday and i need to change it worried i may be on hold until then|
|US Airways|negative        |and when will one of these agents be available to speak
|US Airways|negative        |just hung up on me again another waste of an hour of my time how am i supposed to book a one way award flight
|US Airways|negative        |youre killing me from the inside
|US Airways|negative        |is not the new is more like the new love aa not impressed with the subpar planes and gate agents
+-----+-----+-----+
only showing top 5 rows
SUCCESS: The process with PID 40312 (child process of PID 31596) has been terminated.
SUCCESS: The process with PID 31596 (child process of PID 50480) has been terminated.
SUCCESS: The process with PID 50480 (child process of PID 26948) has been terminated.
PS C:\Users\ONUR\Desktop\big-data-project> 

```

İşlem 2: Veri Zenginleştirme ve Özellik Çıkarımı:

Bu işlemin temel amacı, doğal dil işleme modellerine beslenecek metinsel verilerin anlamsal özniteliklerini artırmak ve veri kümesini yeni yapısal özelliklerle zenginleştirmektir. DataFrame şemasını genişletmek amacıyla Apache Spark'ın 'withColumn()' dönüşüm operatörü kullanılmış ve 'length()' fonksiyonu tetiklenerek, her bir işçi düğümünde bulunan şikâyet metinlerinin karakter uzunlukları paralel olarak hesaplanmıştır. Hesaplanan bu metrikler veri setine "text_length" adında yeni bir sütun olarak enjekte edilmiştir. Bu operasyon, Apache Spark'ın değiştirilemezlik prensibine uygun şekilde, diskteki ham veriyi manipüle etmeden tamamen RAM üzerinde yeni bir veri soy ağacı oluşturarak gerçekleştirilmiştir.

Kod:

```

def add_text_length_column(self) -> None:
    if self.df is None:
        print("Hata: Önce veri setini yüklemelisiniz!")
        return

    transformed_df = (self.df
                      .select("airline", "airline_sentiment",
                              "text")
                      .withColumn("text_length",
                                  length(col("text"))))
    transformed_df.show(5, truncate=False)

```

Çıktı:

```
--- İŞLEM 2 (Transformation): withColumn() ile Sütun Türetme ---  
Açıklama: Tweet metinlerinin uzunluklarının hesaplanıp yeni bir sütuna yazdırılması.
```

airline	airline_sentiment	text	text_length
Virgin America	neutral	what said	9
Virgin America	positive	plus youve added commercials to the experience tacky	52
Virgin America	neutral	i didnt today must mean i need to take another trip	51
Virgin America	negative	its really aggressive to blast obnoxious entertainment in your guests faces amp they have little recourse	105
Virgin America	negative	and its a really big bad thing about it	39

```
only showing top 5 rows  
SUCCESS: The process with PID 65780 (child process of PID 27612) has been terminated.  
SUCCESS: The process with PID 27612 (child process of PID 41688) has been terminated.  
SUCCESS: The process with PID 41688 (child process of PID 32364) has been terminated.  
PS C:\Users\ONUR\Desktop\big-data-project> []
```

İşlem 3: Dağıtık Küme ve Aritmetik Özetleme:

PySpark'taki 'groupBy()' operatörü kullanılarak, sosyal medya etkileşimlerinin şirket bazlı analizi gerçekleştirilmiştir. Küme tabanlı bu işlemde, veriler havayolu şirketlerine göre paralel düğümlerde gruplanmış ve ardından 'agg()' içerisindeki 'avg()' fonksiyonu ile her bir grubun ortalama retweet sayıları hesaplanmıştır. Sonuçların çıktıda daha okunaklı olması amacıyla 'round()' fonksiyonu kullanılarak değerler virgülden sonra iki basamağa yuvarlanmış, 'alias()' ile yeniden isimlendirilmiş ve 'orderBy()' ile büyükten küçüğe sıralanmıştır.

Kod:

```
def calculate_avg_retweets_by_airline(self) -> None:  
    if self.df is None:  
        print("Hata: Önce Veri Setini Yüklelemelisiniz!")  
        return  
    grouped_df = (self.df  
                  .groupBy("airline")  
                  .agg(round(avg("retweet_count"),  
2).alias("avg_retweets"))  
                  .orderBy(col("avg_retweets").desc()))  
  
    grouped_df.show(truncate=False)
```

Çıktı:

```
--- İŞLEM 3 (Transformation): groupBy() ve agg() ile Kümeleme ---  
Açıklama: Havayolu şirketlerine göre ortalama retweet sayılarının hesaplanması.
```

airline	avg_retweets
Delta	0.11
United	0.11
US Airways	0.09
Southwest	0.06
Virgin America	0.05
American	0.04
NULL	NULL

İşlem 4: Dağıtık Tekilleştirme ve Sıralama:

Veri madenciliği süreçlerinde, özellikle Twitter gibi bot hesapların ve spam içeriklerin yoğun olduğu platformlarda veriyi tekilleştirmek modelin sapmasını (bias) engellemek için kritik bir adımdır. Bu işlemde, Spark'ın dağıtık düğümleri üzerinde ‘dropDuplicates(["text"])’ dönüşümü çalıştırılarak tamamen aynı metne sahip olan mükerrer kayıtlar RAM üzerinden temizlenmiştir. Ardından elde edilen özgün veri seti, küresel düzeyde bir sıralama operatörü olan ‘orderBy()’ ile “retweet_count” baz alınarak azalan şekilde sıralanmıştır. Bu sayede platformdaki en yüksek organik etkileşimi alan özgün şikayetler tespit edilmiştir.

Kod:

```
def get_top_unique_retweeted_tweets(self) -> None:
    if self.df is None:
        print("Hata: Önce Veri Setini Yüklemlisiniz!")
        return

    unique_sorted_df = (self.df
                        .dropDuplicates(["text"])
                        .orderBy(col("retweet_count").desc())
                        .select("airline", "retweet_count",
                              "text"))
    unique_sorted_df.show(5, truncate=False)
```

Çıktı:

```
--- İŞLEM 4 (Transformation): dropDuplicates() ve orderBy() ---
Açıklama: Mükerrer tweetlerin silinmesi ve en yüksek retweet alanların sıralanması.

+-----+-----+-----+
|airline|retweet_count|text|
+-----+-----+-----+
|US Airways|44|5 hr flight delay and a delay when we land is that even real life get me off this plane i wanna go home 3 heel clicks|
|US Airways|32|of course never again tho thanks for tweetin ur concern but not doin anythin to fix what happened ill choose wiser next time|
|Delta|31|stop usingthisword if youre a company rt our fleets on fleek|
|US Airways|28|with this livery back in the day|
|Delta|22|can you not rt our fleets on fleek|
+-----+-----+-----+
only showing top 5 rows
SUCCESS: The process with PID 27412 (child process of PID 57628) has been terminated.
SUCCESS: The process with PID 57628 (child process of PID 21552) has been terminated.
SUCCESS: The process with PID 21552 (child process of PID 45348) has been terminated.
PS C:\Users\ONUR\Desktop\big-data-project> []
```

6.3 Dağıtık Veri Eylem (Action) Operasyonları:

İşlem 1: Toplam Kayıt Sayımı:

Büyük veri setlerinin analizinde veri bütünlüğünü doğrulamak amacıyla ‘count()’ eylem operatörü kullanılmıştır. Bu komut tetiklendiğinde, Spark kümesindeki tüm işçi düğümleri kendi belleklerindeki veri parçalarını saymış ve sonuçları toplayarak ana makineye toplam kayıt sayısını döndürmüştür.

Kod:

```
def count_total_records(self) -> None:
    if self.df is None:
        print("Hata: Önce Veri Setimi Yüklemlisiniz!")
        return
    total_rows = self.df.count()
    print(f"Veri setindeki toplam kayıt sayısı: {total_rows}")
```

Çıktı:

```
--- İŞLEM 1 (Action): count() ---
Açıklama: Dağıtık DataFrame üzerindeki toplam kayıt sayısının hesaplanması.

Veri setindeki toplam kayıt sayısı: 14623
```

İşlem 2: Dağıtık İstatistiksel Özetleme:

Veri setindeki sayısal özelliklerin genel dağılımını anlamak için 'describe()' operatörü çalıştırılmıştır. Bu eylem sayesinde 'retweet_count' sütunu üzerindeki standart sapma, ortalama, minimum ve maksimum değerler paralel olarak hesaplanıp tablo formatında sunulmuştur.

Kod:

```
def describe_numerical_columns(self) -> None:
    if self.df is None:
        print("Hata: Önce Veri Setini Yüklemlisiniz!")
        return
    desc_df = self.df.describe(["retweet_count"])
    desc_df.show()
```

Çıktı:

```
--- İŞLEM 2 (Action): describe() ---
Açıklama: 'retweet_count' sütununun istatistiksel özetinin çıkarılması.

+-----+-----+
|summary|retweet_count|
+-----+-----+
|count|14604|
|mean|0.08278553820870994|
|stddev|0.7466472933512464|
|min|0|
|max|44|
+-----+-----+
```

İşlem 3: İşçi Düzgümlerinden Veri Toplama:

Platformdaki en yüksek etkileşimi alan 5 tweet, dağıtık bellekten 'collect()' eylem operatörü ile çekilmiş ve yerel bir Python listesine dönüştürülerek raporlanmıştır. Ağ darboğazı ve Out-Of-Memory hatalarını önlemek adına collect öncesinde 'limit(5)' operatörüyle sıkı bir kısıtlama uygulanmıştır.

Kod:

```
def collect_top_retweeted(self) -> None:
    if self.df is None:
        print("Hata: Önce veri setini yüklemelisiniz!")
        return
    top_tweets_list = (self.df
                        .select("airline", "retweet_count", "text")
                        .orderBy(col("retweet_count").desc())
                        .limit(5)
                        .collect())

    print("ÇEKİLEN EN YÜKSEK ETKİLEŞİMLİ 5 TWEET]")
    for i, row in enumerate(top_tweets_list, 1):
        print(f"{i}. Şirket: {row['airline']} | Retweet: {row['retweet_count']}")
        print(f"    Metin: {row['text'][:80]}...\n")
```

Çıktı:

```
--- İŞLEM 3 (Action): collect() ---
Açıklama: En yüksek retweet alan 3 tweetin Driver (Ana Makine) belleğine çekilmesi.

ÇEKİLEN EN YÜKSEK ETKİLEŞİMLİ 5 TWEET]
1. Şirket: US Airways | Retweet: 44
   Metin: 5 hr flight delay and a delay when we land is that even real life get me off thi...

2. Şirket: US Airways | Retweet: 32
   Metin: of course never again tho thanks for tweetin ur concern but not doin anythin to ...

3. Şirket: Delta | Retweet: 31
   Metin: stop usingthisword if youre a company rt our fleets on fleek...

4. Şirket: US Airways | Retweet: 28
   Metin: with this livery back in the day...

5. Şirket: Delta | Retweet: 22
   Metin: can you not rt our fleets on fleek...
```

6.4 Dağıtık Spark SQL Kullanımı:

İşlem 1 ve 2: Sanal SQL Tablosu Entegrasyonu ve Temel Sorgulama:

Mevcut dağıtık DataFrame yapısı üzerinde doğrudan SQL sorgusu çalıştırabilmek için öncelikle verinin bir tablo olarak deklare edilmesi gerekmektedir. Bu bağlamda

'createOrReplaceTempView("tweets_view")' (İşlem 1) metodu kullanılarak RAM üzerinde sanal bir ilişkisel tablo yansıması oluşturulmuştur. Bu hazırlığın ardından 'spark.sql()' fonksiyonuna (İşlem 2) standart SQL syntax'ı enjekte edilmiş. Bu sorgu ile en çok şikâyet alan şirketler getirilmiştir.

Kod:

```
def execute_spark_sql_basic(self) -> None:
    if self.df is None:
        print("Hata: Önce veri setini yüklemelisiniz!")
        return
    # 1. işlem
    self.df.createOrReplaceTempView("tweets_view")
    # 2. işlem
    query = """
        SELECT airline, COUNT(*) AS total_negative_feedback
        FROM tweets_view
        WHERE airline_sentiment = 'negative'
        GROUP BY airline
        ORDER BY total_negative_feedback DESC
    """
    sql_df = self.spark.sql(query)
    sql_df.show(truncate=False)
```

Çıktı:

--- İŞLEM 1 ve 2 (Spark SQL): Temp View ve Temel Sorgu ---
Açıklama: Verinin SQL tablosuna çevrilip en çok şikâyet alan şirketlerin bulunması.

airline	total_negative_feedback
United	2633
US Airways	2263
American	1941
Southwest	1186
Delta	955
Virgin America	181

İşlem 3: İleri Düzey SQL Analitiği ve Window Fonksiyonları:

Büyük veri analitiğinde, veriyi sadece genel olarak kümelemek değil, her bir kümenin kendi içerisindeki hiyerarşiyi bulmak (örneğin "Her kategorinin en iyisi") ciddi bir algoritmik maliyet yaratır. Bu problemi minimum maliyetle çözmek için SQL'in en gelişmiş özelliklerinden biri olan Window Fonksiyonları kullanılmıştır. Sorgu içerisinde 'ROW_NUMBER() OVER(PARTITION BY airline ORDER BY retweet_count DESC)' mimarisi kurularak, sistemdeki tüm tweetler havayolu şirketlerine göre alt pencerelere bölünmüştür. Her pencere kendi içinde retweet sayılarına göre

sıralanıp numaralandırılmış, ardından iç içe SQL mantığıyla sadece 1 numaralı sıraya yerleşen (yani kendi şirketinin en çok etkileşim alan) şikâyet kayıtları sisteme çekilmiştir. Bu işlem, Spark'ın analitik zekasını ve ileri düzey SQL yorumlama kapasitesini kanıtlamaktadır.

Kod:

```
def execute_spark_sql_advanced(self) -> None:
    if self.df is None:
        print("Hata: Önce Veri Setini Yüklemelisiniz!")
        return

    self.df.createOrReplaceTempView("tweets_view")
    # Mantık: Önce şirketlere göre (PARTITION BY) grupla,
    retweet'e göre (ORDER BY) sırala,
    # her birine sıra numarası (rn) ver. Dış sorguda ise sadece
    sıra numarası 1 olanları getir!
    query = """
        SELECT airline, retweet_count, text
        FROM (
            SELECT airline, retweet_Count, text,
                   ROW_NUMBER() OVER(PARTITION BY airline ORDER BY
retweet_count DESC) as rn
            FROM tweets_view
        ) ranked_tweets
        WHERE rn = 1
        ORDER BY retweet_Count DESC
    """

    sql_df = self.spark.sql(query)
    sql_df.show(truncate=False)
```

Çıktı:

--- İŞLEM 3 (Spark SQL): İleri Düzey Window Functions ---
Açıklama: Her bir havayolu şirketinin en çok etkileşim alan şikayetinin bulunması.

airline	retweet_count	text
US Airways	44	5 hr flight delay and a delay when we land is that even real life get me off this plane i wanna go home 3 heel clicks
Delta	31	stop usingthisword if youre a company rt our fleets on fleek
Southwest	22	beautiful day in seattle
United	7	but i got a bad exchange rate so want my money back please can you refund without penalty
American	5	cancelled flight my flight saying it is cause of weather condition when it was the only flight that was cancelled flight
Virgin America	4	can u help with flights to get a 2y old battling cancer who needs treatment in
NULL	NULL	NULL

SUCCESS: The process with PID 7632 (child process of PID 14980) has been terminated.
SUCCESS: The process with PID 14980 (child process of PID 30944) has been terminated.
SUCCESS: The process with PID 30944 (child process of PID 44420) has been terminated.
PS C:\Users\ONUR\Desktop\big-data-project> []

6.5 Pandas ve PySpark Karşılaştırması:

Bu işlemin temel amacı, tekil düğüm bellek mimarisinde çalışan Pandas kütüphanesi ile dağıtık küme mimarisine sahip PySpark motoru arasındaki yürütme hızlarını kıyaslayarak büyük veri sistemlerinin ölçeklenebilirlik dinamiklerini analiz etmektir. Test kapsamında, aynı temizlenmiş veri seti üzerinden havayolu şirketlerine göre gruplama yapılmış; ortalama retweet ve toplam şikâyet sayıları hesaplanarak çalışma süreleri milisaniye bazında ölçülmüştür.

Kod:

```
def test_pandas(self) -> float:
    start_time = time.time()
    pandas_df = pd.read_csv(self.file_path)
    result = pandas_df.groupby('airline').agg(
        avg_retweets=('retweet_count', 'mean'),
        total_feedbacks=('airline_sentiment', 'count'))
    end_time = time.time()
    return (end_time-start_time) * 1000

def test_pyspark(self) -> float:
    start_time = time.time()
    spark_df = (self.spark.read
                .option("header", "true")
                .option("inferSchema", "true")
                .csv(self.file_path))
    result = spark_df.groupBy("airline").agg(
        avg("retweet_count").alias("avg_retweets"),
        count("airline_sentiment").alias("total_feedbacks")
    )
```

```

        result.collect()
        end_time = time.time()
        return (end_time - start_time) * 1000

def run_comparison(self) -> None:
    pandas_time = self.test_pandas()
    spark_time = self.test_pyspark()
    print("\n --- SONUÇLAR --- ")
    print(f"Pandas Çalışma Süresi : {pandas_time:.2f} ms")
    print(f"PySpark Çalışma Süresi : {spark_time:.2f} ms\n")

```

Çıktı:

```

--- Pandas vs PySpark Performans Karşılaştırması ---
Açıklama: Aynı Aggregate (Kümeleme) işleminin iki farklı motorda yarışdırılması.

Pandas testi çalışıyor...
PySpark testi çalışıyor...

--- SONUÇLAR ---
Pandas Çalışma Süresi : 38.84 ms
PySpark Çalışma Süresi : 3444.85 ms

```

Değerlendirme:

Simülasyon sonucunda, yerel ortamda Pandas kütüphanesinin PySpark'a kıyasla işlemi çok daha kısa bir sürede tamamladığı gözlemlenmiştir. Bu durumun temel nedenleri şunlardır:

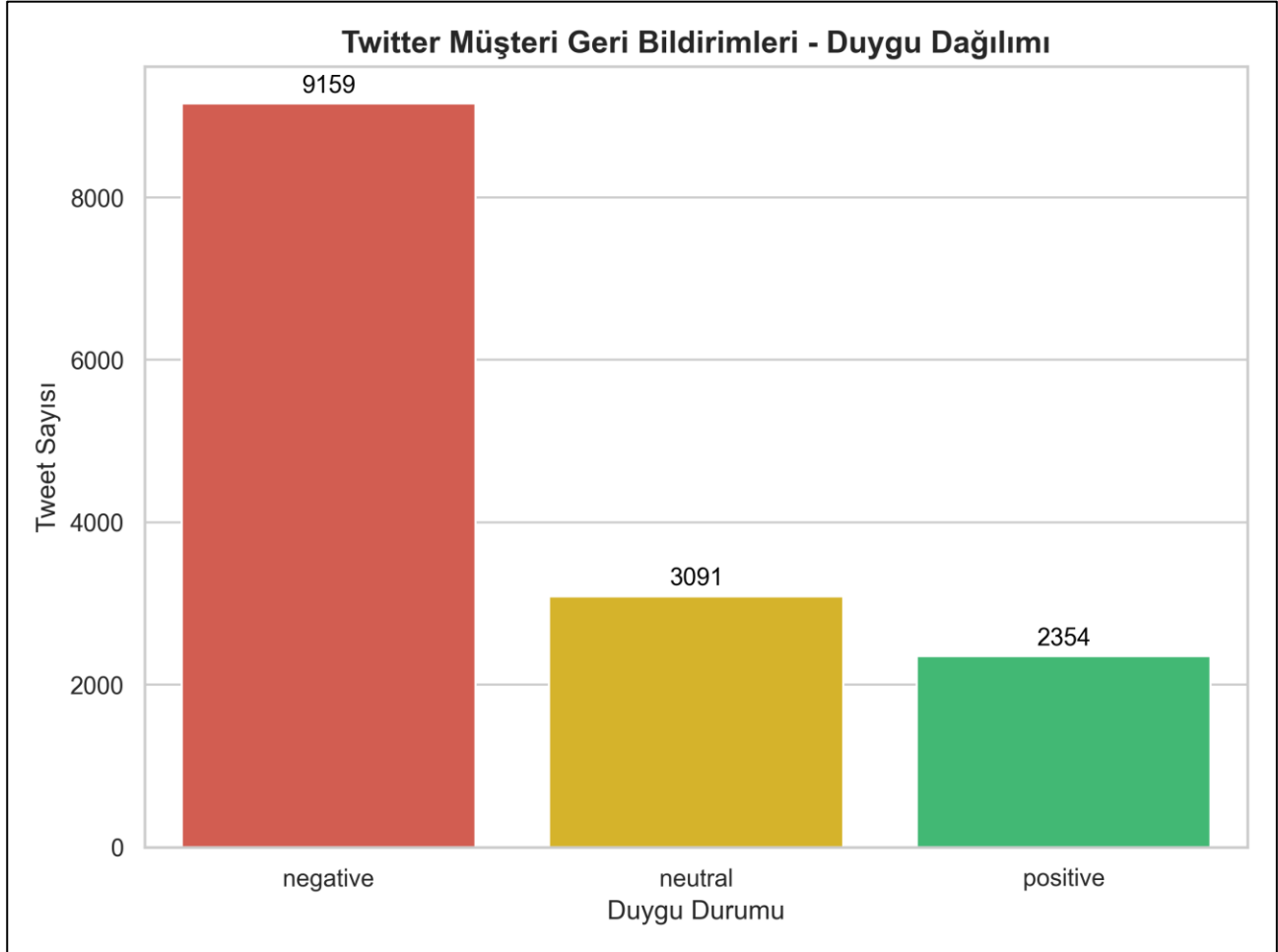
1. **Bellek ve Veri Ölçeği:** Projede işlenen 14.604 satırlık veri kümesi, modern sistemler için "Mikro Veri" sınıfındadır ve yerel bilgisayarın RAM kapasitesine tamamen sığmaktadır. Pandas, veriyi tek bir thread üzerinden doğrudan ardışık okuyarak bellekte hızla işler.
2. **Spark Altyapı Maliyeti:** PySpark, bu küçük veri kümesini işlemek için bile arka planda Java Sanal Makinesini (JVM) tetikler, Yönlendirilmiş Döngüsüz Grafik (DAG) yürütme planı hazırlar ve görevi worker threads paylaşdırmak için serileştirme işlemlerine katlanır. Küçük verilerde bu koordinasyon ve iletişim süresi, asıl hesaplama süresinden çok daha uzun sürer.

Bu karşılaştırma sonucunda, teknolojilerin saf hızlarına göre değil, veri hacmine göre seçilmesi gerektiğini kanıtlanmıştır.

7. Veri Görselleştirme:

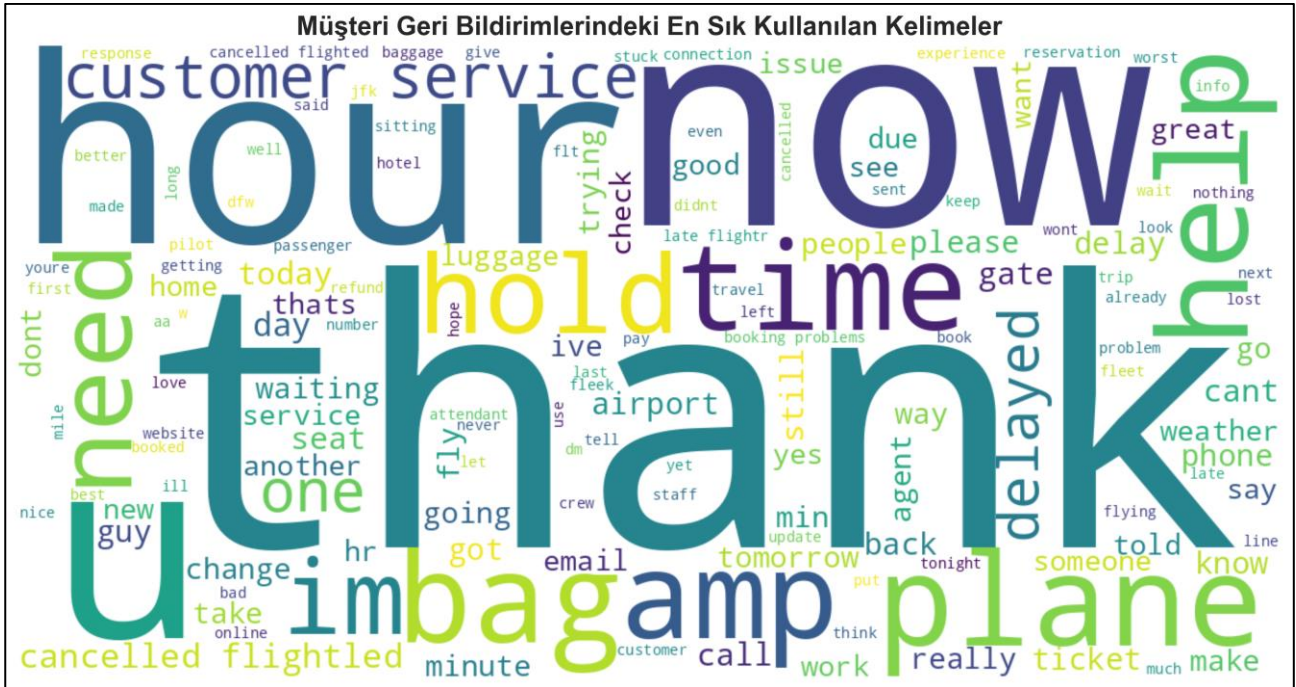
Projenin bu aşaması, elde edilen analitik verilerin, yöneticiler ve karar alıcılar için iş zekası standartlarında yorumlanabilmesi amacıyla Python'ın matplotlib, seaborn ve wordcloud kütüphaneleri kullanılarak görselleştirme adımları gerçekleştirilmiştir.

Grafik 1: Genel Duygu Dağılımı:



Grafik, sosyal medya platformunda havayollarına yönelik atılan tüm tweetlerin genel duygu (sentiment) eğilimini yansıtmaktadır. Sonuçlar incelendiğinde, "Negatif" geri bildirimlerin pozitif ve nötr kayıtlara kıyasla ezici bir çoğunluğa sahip olduğu açıkça görülmektedir. Bu dağılım, müşterilerin Twitter'ı marka sadakati oluşturmaktan ziyade bir "dijital şikayet mecrası" olarak kullandığını doğrulamaktadır. Markaların sosyal medya kriz yönetimi stratejilerini yeniden gözden geçirmeleri gerektiğine işaret eder.

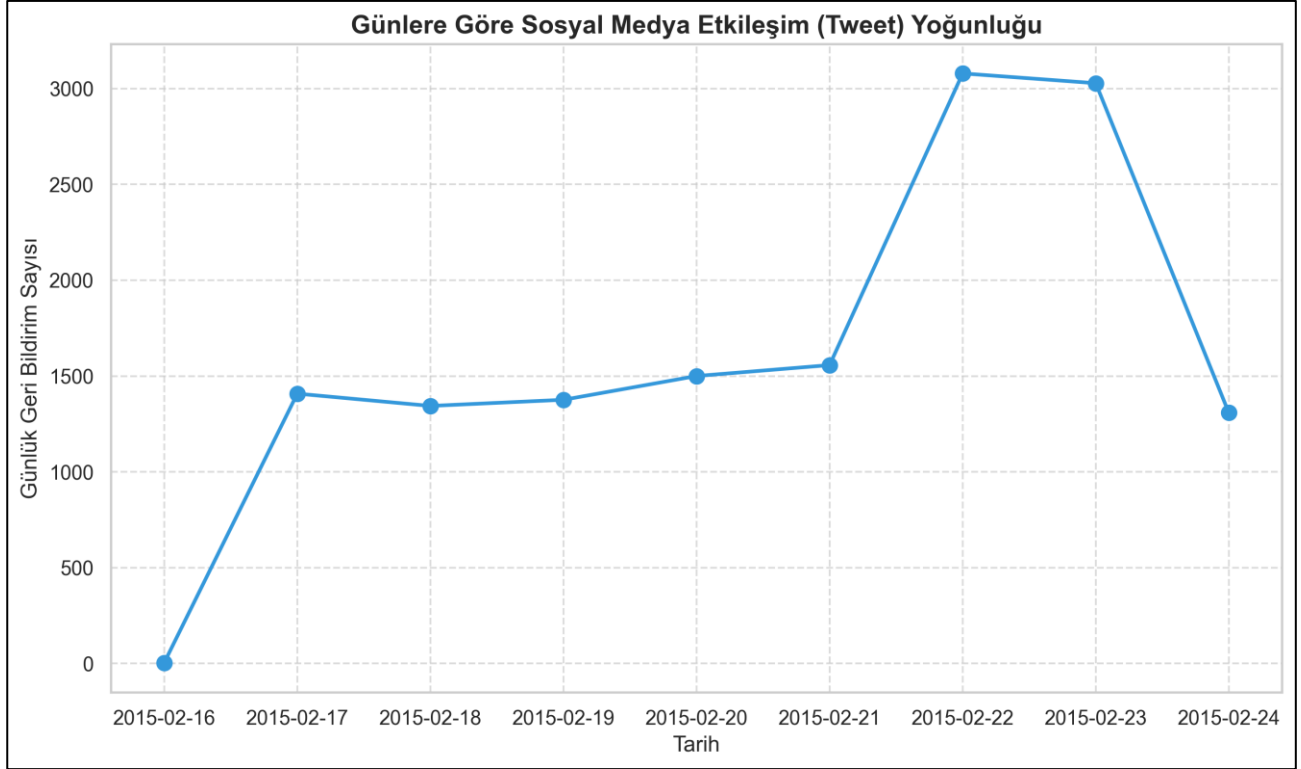
Grafik 2: En Sık Kullanılan Kelimeler:



Bu Kelime Bulutu, müşteri şikayetlerinin anlamsal yoğunluğunu göstermektedir. Analizin doğruluğunu artırmak amacıyla, veri setinde kaçınılmaz olarak çok fazla geçen "flight" ve "airline" gibi jenerik kelimeler özel filtreler (stopwords) ile temizlenmiş ve gerçek sorunlar (kök nedenler) gün yüzüne çıkarılmıştır.

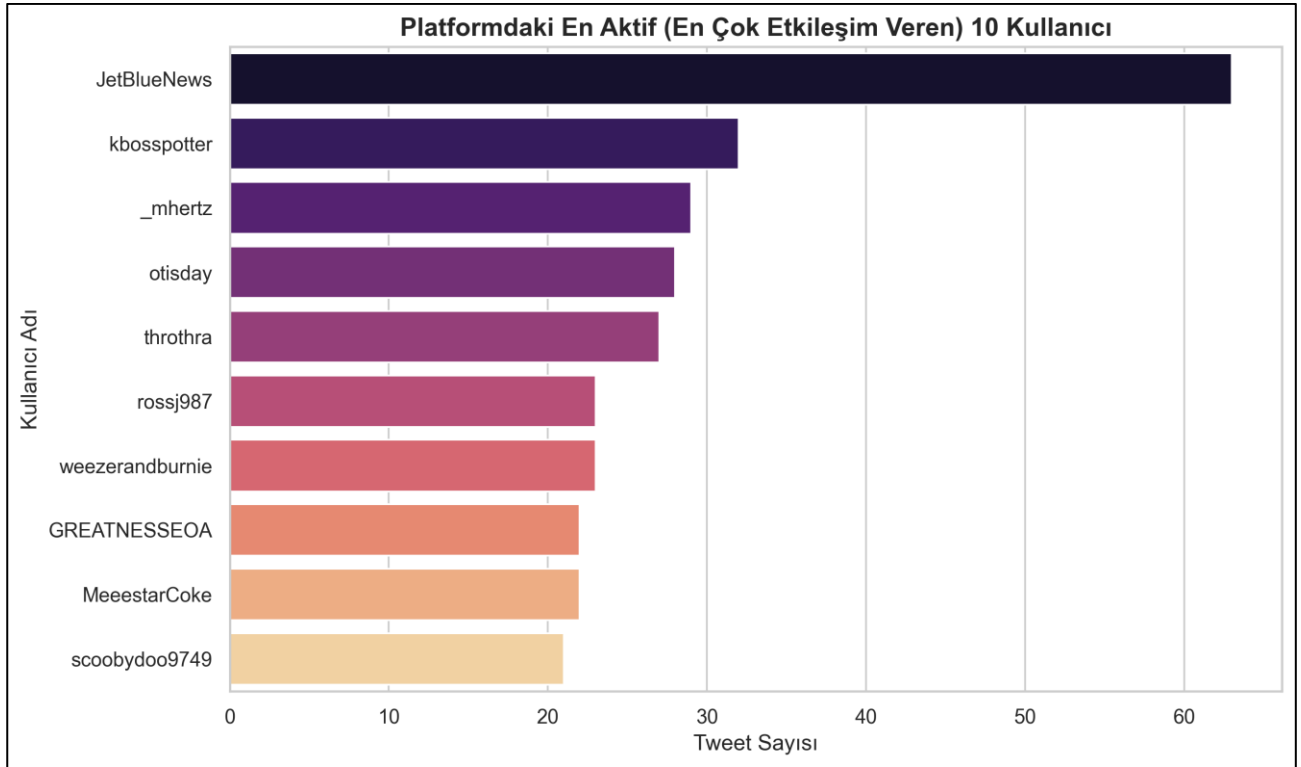
Görsel incelendiğinde; "cancelled", "delay", "customer service", "hour" ve "hold" kelimelerinin en büyük puntolarla öne çıktığı görülmektedir. Bu sonuç, binlerce metni tek tek okumaya gerek kalmadan, müşterilerin temel acı noktalarının saatler süren operasyonel rötarlar, son dakika iptalleri ve çağrı merkezlerinde çözümsüz bırakılmaları olduğunu doğrudan ispatlamaktadır.

Grafik 3: Zaman Serisi Analizi - Günlük Etkileşim Yoğunluğu:



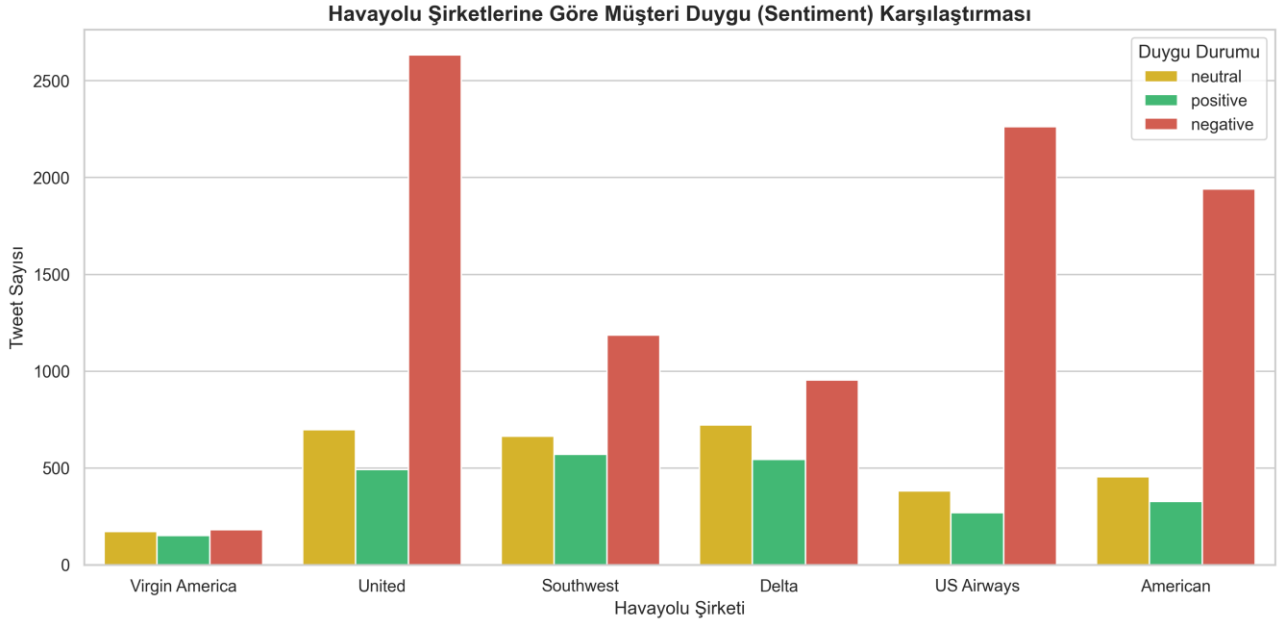
Bu zaman serisi (Time Series) grafiği, ‘tweet_created’ alanı referans alınarak platformdaki müşteri geri bildirimlerinin günlere göre gösterdiği dalgalanmaları ve frekans değişimlerini analiz etmektedir. Çizgi grafik üzerindeki keskin yükseliş (peak) noktaları, ilgili günlerde havayolu operasyonlarında sistemsel bir kriz (yaygın uçuş iptalleri, kötü hava koşulları gibi) yaşandığını ve müşteri tepkilerinin sosyal medyada yoğunlaştığını net bir şekilde ortaya koymaktadır. Elde edilen bu metrik, kriz anlarının geriye dönük tespiti ve operasyonel verimsizliklerin tarihsel analizi için kritik bir iş zekâsı (BI) çıktısı sağlamaktadır.

Grafik 4: En Aktif Kullanıcılar (Top 10):



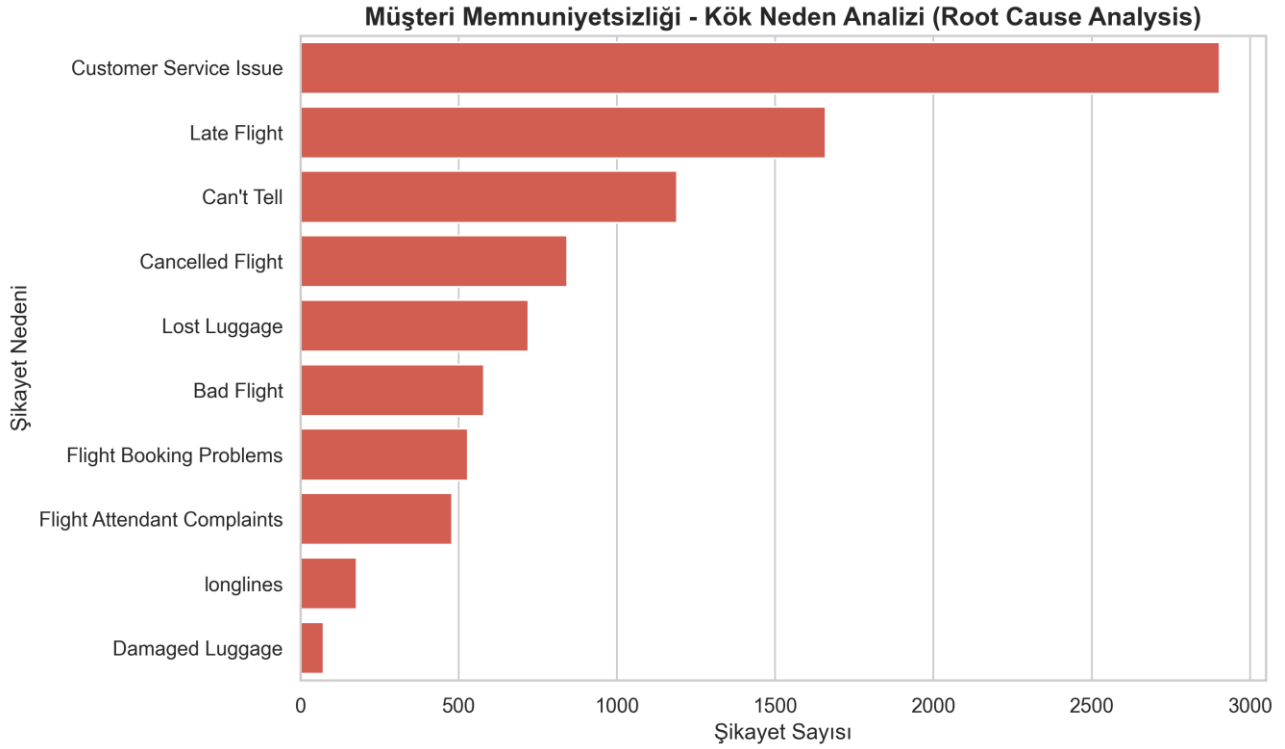
Bu yatay bar grafik, platformda havayolu şirketlerine yönelik en yüksek etkileşimi gerçekleştiren ilk 10 aktif kullanıcıyı listelemektedir. Sosyal ağ analizi (SNA) perspektifinden bakıldığında; bu kullanıcılar markanın sadık savunucuları olabileceği gibi, sistematik olarak olumsuz algı üreten bot veya trol hesaplar da olabilir. Dolayısıyla, bu yüksek etkileşimli hesapların detaylı bir şekilde analiz edilmesi, markanın itibar yönetimi süreçleri için öncelikli bir aksiyon planı niteliğindedir.

Grafik 5: Havayolu Şirketlerine Göre Duygu Kırılımı:



Hangi havayolunun müşteri ilişkilerinde daha başarılı olduğunu gösteren bu çok değişkenli kümelenmiş bar grafiği, rekabet analizi için kritik bir İş Zekâsı çıktısıdır. Grafik incelendiğinde; United, US Airways ve American Airlines şirketlerinin devasa oranda negatif geri bildirim alarak kriz yaşadığı, buna karşılık Virgin America ve Delta'nın negatif/pozitif oranlarının birbirine daha yakın olduğu ve nispeten daha başarılı bir müşteri memnuniyeti stratejisi yürüttükleri görülmektedir.

Grafik 6: Müşteri Memnuniyetsizliği Kök Neden Analizi:



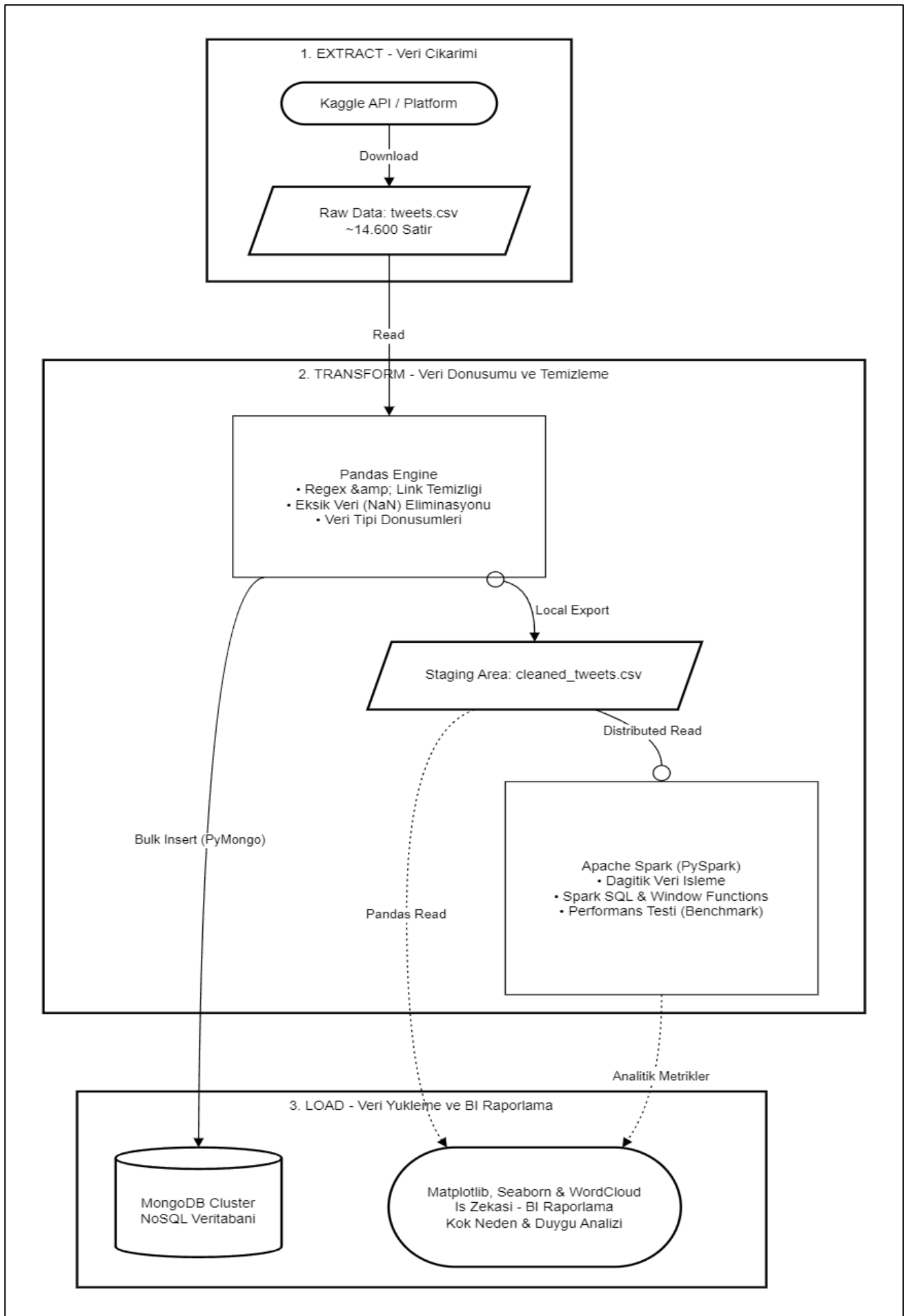
Bir önceki Kelime Bulutu grafiğinde keşfedilen anlamsal sorunları yapısal ve ölçülebilir bir veriye döken bu görsel, yalnızca "Negatif" etiketli tweetleri izole etmektedir. Tabloya göre, sektörel bazda müşterileri en çok rahatsız eden kök problemin açık ara farkla "Müşteri Hizmetleri (Customer Service Issue)" olduğu tespit edilmiştir. Bunu operasyonel sorunlar olan "Rötarlar (Late Flight)" izlemektedir. Karar alıcı bir yönetici için bu grafik, operasyonel bütçenin doğrudan müşteri destek ekiplerinin eğitime ve altyapısına kaydırılması gerektiğini sayısal olarak ispatlamaktadır.

8. ETL Diyagramı ve Dokümantasyon:

Bu aşamada, projenin ham veri kaynağından başlayarak nihai veri tabanı katmanına ve iş zekâsı raporlama araçlarına kadar uzanan tüm veri hattı (data pipeline) mimarisi Veri Mühendisliği standartlarında belgelenmiştir. Proje kapsamında uçtan uca kurgulanan sistem mimarisi, veri akış yönü ve kullanılan teknolojilerin rolleri teorik temelleriyle ele alınmıştır.

8.1 Uçtan Uca ETL Mimari Akış Diyagramı:

Projenin veri akış hattını, bileşenler arası ilişkileri ve kullanılan veri işleme motorlarını gösteren şema aşağıda yer almaktadır.



Diyagramda Modellenen Akışın Teknik Açıklaması:

1. **EXTRACT (Veri Çıkarımı) Katmanı:** Veri hattının başlangıç noktasıdır. Kaggle platformu üzerinde açık kaynak olarak sunulan ham Twitter Müşteri Geri Bildirimleri verisi (tweets.csv), yaklaşık 14.600 satır ve yapılandırılmamış metin formatında yerel çalışma ortamına aktarılmıştır.
2. **TRANSFORM (Veri Dönüşümü ve Temizleme) Katmanı:** Projenin çekirdek hesaplama katmanıdır. Kendi içinde iki alt motora ayrılmıştır:
 - **Pandas Engine:** Bellek üzerinde hızlı ön işlem adımları için kullanılmıştır. Düzenli ifadeler (Regex) kullanılarak metinlerdeki gürültüler (linkler, @mention ifadeleri) temizlenmiş, eksik veriler (NaN) elenmiş ve veri tipleri şemaya uygun hale getirilmiştir. Temizlenen veri bir ara katman olarak 'cleaned_tweets.csv' dosyasına yazılmıştır.
 - **Apache Spark Engine (PySpark):** Büyük veri standartlarında dağıtık veri işleme kabiliyeti için ara katmandaki veri PySpark DataFrame yapısına alınmıştır. Spark SQL, filtreleme, gruptama ve Window Functions operasyonları ile analitik metrikler dağıtık küme mimarisine uygun olarak hesaplanmıştır.
3. **LOAD (Veri Yükleme) Katmanı:** Dönüştürülen verilerin kalıcı hale getirildiği ve sunulduğu son katmandır. Pandas tarafından temizlenen yapısal veri, NoSQL mimarisine uygun olarak MongoDB Cluster yapısına doküman tabanlı olarak toplu şekilde yüklenmiştir. Eş zamanlı olarak PySpark ve Pandas katmanlarından elde edilen analitik çıktılar, Matplotlib, Seaborn ve WordCloud kütüphanelerine beslenerek karar alıcılar için İş Zekası raporlarına dönüştürülmüştür.

8.2 ETL ve ELT Yaklaşımlarının Karşılaştırması:

Modern veri mimarilerinde verinin kaynaktan hedef sisteme taşınması ve işlenmesi sürecinde benimsenen iki temel paradigma bulunmaktadır: ETL (Extract, Transform, Load) ve ELT (Extract, Load, Transform). Bu iki yaklaşım arasındaki temel farklar aşağıdaki tabloda detaylandırılmıştır:

Kriter	ETL	ELT
İşlem Sırası	Veri çıkarılır, dönüştürülür ve temizlendikten sonra hedefe yüklenir.	Veri çıkarılır, ham haliyle hedefe yüklenir ve dönüşüm hedef içinde yapılır.

Dönüşüm Yeri	Hedef sistemin dışında, harici bir işleme motorunda (Spark / Pandas) gerçekleşir.	Doğrudan hedef veri tabanının veya bulut veri ambarının içinde gerçekleşir.
Veri Kalitesi	Hedef sisteme yalnızca temizlenmiş, yüksek kaliteli ve yapısal veri girer.	Ham ve kirli veri doğrudan veri tabanına girer. Temizlik ve anlamlandırma sorgularla içeride yapılır.
Teknoloji Odağı	Harici işlem motorları ve dağıtık hesaplama araçları ön plandadır.	Güçlü bulut veri ambarları ve veri gölleri ön plandadır.
Depolama Maliyeti	Sadece yapısal veri tutulduğu için hedef sistemde daha az depolama alanı gerektirir.	Ham verinin tamamı saklandığı için yüksek depolama kapasitesi gerektirir.

8.3 Proje Mimarisinin Gerekçelendirilmesi (Neden ETL?)

Bu proje kapsamında kurgulanan veri hattı mimarisi, teorik ve pratik gerekçeleriyle birlikte kesinlikle ETL (Extract, Transform, Load) yaklaşımına dayanmaktadır. Bu mimari tercihin gerekçeleri projedeki somut adımlarla şu şekilde açıklanabilir:

- 1. Verinin Ön İşleme Adımından Geçmesi:** Kaggle'dan çekilen ham veriler doğrudan hedef veritabanı olan MongoDB'ye yüklenmemiştir. Veri henüz dış ortamdayken, Python'ın Pandas motoru kullanılarak gürültülerden arındırılmış, Regex filtreleri uygulanmış ve yapısal bir forma sokulmuştur. Hedef sistem olan veri tabanına (MongoDB) sadece bu temizleme işleminden geçmiş olan cleaned_tweets.csv verisi aktarılmıştır. Bu durum, sürecin klasik bir ETL hattı olduğunu gösterir.
- 2. Hesaplama Yükünün Dağıtılması ve Veritabanı Optimizasyonu:** Dönüşüm işlemlerinin ağır analitik kısımları (gruplamalar, zaman serisi analizleri, pencere fonksiyonları) MongoDB sorguları üzerinde değil, harici ve güçlü bir dağıtık işlem motoru olan PySpark üzerinde gerçekleştirilmiştir. Eğer ELT yaklaşımı seçilseydi, ham veri doğrudan MongoDB'ye yüklenecek ve tüm bu metin madenciliği, temizlik ve aggregation işlemleri NoSQL sorgularıyla veri tabanının kendi kaynakları tüketilerek yapılacaktı. Harici işlem motorları (Pandas & Spark) kullanılarak veri tabanı yorulmamış ve sistem kaynakları optimize edilmiştir.
- 3. Veri Kirliliğinin Önlenmesi:** Twitter verileri doğası gereği yüksek oranda gürültü, çoklu boş değer (NaN) ve tutarsız metinler içerir. ETL yaklaşımı sayesinde bu gürültülü verilerin

kalıcı veri tabanı şemasına sızması engellenmiş, MongoDB içerisinde tutarlı, temiz ve doğrudan sorgulanabilir doküman koleksiyonları oluşturulmuştur.

Sonuç olarak; harici işlem alanlarının aktif kullanımı, veri tabanına yazılmadan önce uygulanan katı temizlik kuralları ve analitik hesaplamaların Spark mimarisiyle dışarıda koşturulması projenin saf bir ETL pipeline mimarisiyle tasarlandığını kesin olarak kanıtlamaktadır.

9. Kaynakça:

- Kaggle — Twitter US Airline Sentiment Dataset: kaggle.com/datasets/crowdflower/twitter-airline-sentiment
- MongoDB Documentation: docs.mongodb.com
- Apache Spark Documentation: spark.apache.org/docs
- Pandas Documentation: pandas.pydata.org
- Matplotlib Documentation: matplotlib.org
- Gemini